IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 21 October 2024, accepted 22 November 2024, date of publication 27 November 2024, date of current version 9 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3507215

SURVEY

Securing the Shared Kernel: Exploring Kernel Isolation and Emerging Challenges in Modern Cloud Computing

SEHAR ZEHRA[1], HASSAN JAMIL SYED[2], FAHAD SAMAD[3], UMMAY FASEEHA[1], HAMZA AHMED[1], AND MUHAMMAD KHURRAM KHAN[4], (Senior Member, IEEE)

[1]Department of Computer Science, National University of Computer and Emerging Sciences (FAST), Karachi 75030, Pakistan
[2]Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur 57000, Malaysia
[3]Department of Cyber Security, National University of Computer and Emerging Sciences (FAST), Karachi 75030, Pakistan
[4]Center of Excellence in Information Assurance (CoEIA), King Saud University, Riyadh 11451, Saudi Arabia

Corresponding author: Hassan Jamil Syed (hassan.jamil@apu.edu.my)

This work was supported in part by the Asia Pacific University of Technology and Innovation (APU), Kuala Lumpur, Malaysia.

ABSTRACT Containerization is a rapidly advancing technology in cloud computing, facilitating the seamless development, deployment, and management of applications across diverse computing environments. This technology offers lightweight operations, portability, efficiency, and scalability advantages, applicable to developer workstations, mission-critical web servers, and the public cloud. However, unlike Virtual Machines (VMs), containers share the underlying machine's operating system (OS) kernel, which introduces unique security challenges alongside speed and efficiency benefits. These challenges include the risks of container escape attacks, privilege escalation, and exploitation of kernel vulnerabilities. This paper comprehensively reviews state-of-the-art containerization security solutions, focusing on various kernel isolation approaches. It proposes a thematic taxonomy of containerization security, highlighting essential parameters to help developers understand the security needs within a shared kernel environment. This paper describes the current landscape of container security by examining critical developments, challenges, and trends in the existing literature—from system calls to kernel isolation. Additionally, it identifies open research issues and discusses industry best practices and emerging developments in container security, aiming to guide future research and implementation strategies.

INDEX TERMS Containerization, kernel isolation & security, shared kernel environment, cloud computing security, container escape attacks, privilege escalation, virtual machine (VM), system call filtering.

I. INTRODUCTION

In the rapidly evolving world of cloud computing, containerization has become one of the most pivotal technologies, transforming the way applications are built, deployed, and managed. Containers provide a lightweight, portable, and efficient way to package applications and their dependencies. This enables developers to build, deploy, and manage software across diverse computing environments with unprecedented ease and flexibility [1], [2], [3]. Kubernetes and Docker are two of the most popular containerization platforms. They have revolutionized how software is built and implemented in cloud and enterprise environments. Docker allows programmers to package applications and their dependencies into compact, portable containers. Docker streamlines the development and deployment process by enabling developers to create reproducible environments across different computing environments using container images. Additionally, container orchestration platforms like Kubernetes efficiently manage large-scale containerized workloads, enabling organizations to build and run complex distributed systems with unparalleled performance and reliability [4], [5], [6], [7].

Containers possess isolation, portability, efficiency, flexibility, scalability, consistency, DevOps integration, and resource efficiency. These attributes make them the preferred

The associate editor coordinating the review of this manuscript and approving it for publication was Amjad Mehmood.

choice for modern software development and deployment. The adoption of containerization has revolutionized software development practices by facilitating rapid iteration and deployment of applications, encouraging collaboration between development teams, and improving the time-to-market for new features and services [8], [9], [10].

Despite the numerous benefits containers offer for software development and deployment, they face unique security challenges, particularly in environments with shared kernels. In a shared kernel environment, multiple containers run on a single host machine and share access to the underlying host operating system's kernel. Unlike Virtual Machines (VMs), which involve the virtualization of system hardware, including the CPU, memory, storage, and network interfaces, container virtualization operates at the Operating System (OS) level. This means containers share the host OS kernel, contributing to their lightweight nature and enabling faster start and stop times compared to traditional VMs [11], [12], [13], [14].

Container isolation mechanisms, such as Linux namespaces and control groups (cgroups), help create boundaries between containers. This allows them to operate independently while sharing the same kernel environment. While this shared kernel architecture provides a fast and efficient environment for software developers, it also introduces a potential attack surface, exposing systems to unique security vulnerabilities [15], [16], [17].

Shared kernel environments introduce inherent security risks due to the containers' shared access to the underlying operating system kernel. One of the primary challenges in shared kernel environments is the potential for container escape attacks. In these attacks, an attacker escalates their privileges from a container's isolated environment to gain access to the host OS kernel and other containers. This poses a significant security concern [18], [19], [20]. Moreover, shared kernel environments are susceptible to privilege escalation attacks, where an attacker exploits vulnerabilities in the container runtime or kernel to elevate their privileges within the container or gain access to sensitive resources on the host system. Once compromised, attackers can modify applications, compromise data, and expand their access within the containerized system. Such intrusions may result in data breaches, interrupted services, and significant harm to an organization's reputation [21], [22], [23].

Isolation and security are fundamental considerations in containerization because a compromised container can potentially impact other containers running on the same host. This underscores the importance of robust security measures to protect each containerized environment. Therefore, ensuring strong isolation and security measures is crucial to prevent any adverse effects on neighboring container instances and to maintain the integrity of the overall system. Various security vulnerabilities can be effectively defended against at the kernel level using strategies like seccomp, control groups (cgroups), and namespaces. These tools help to restrict and isolate the capabilities of container processes, enhancing the security of the system [24], [25], [26].

Therefore, implementing effective security procedures is crucial to reduce the risks of privilege escalation, container escape attacks, and other security threats that could occur in multi-tenant container installations. By establishing robust security processes and adhering to best practices, organizations can strengthen the overall security posture of their containerized environments and prevent security breaches [27]. This study comprehensively surveys the current state-of-the-art research in containerization, identifying key developments, challenges, and trends in the kernel isolation approach. Although several studies, such as those by Oluyede et al. [25], Minna and Massacci [28], Wong et al. [29], Kawanishi et al. [30], Zhu et al. [31], Javed and Toor [32], and Compastié et al. [33], have addressed aspects of container security, they often do not fully cover the range of challenges associated with shared kernel architectures. This study aims to provide researchers, practitioners, and policymakers with a deep understanding of state-of-the-art kernel isolation strategies for securing containers in shared kernel environments. It synthesizes and analyzes the body of knowledge on this topic, offering insights into potential directions for future research and development. Furthermore, it examines various aspects of containerization security, including threat models, attack vectors, isolation techniques, and security frameworks, assessing the effectiveness of existing solutions in mitigating security risks. The main contributions of this paper are outlined as follows:

i) Surveying the latest research on containerization security, providing in-depth technical and practical insights.

ii) Introducing a thematic taxonomy to categorize existing literature into coherent parameters, enhancing understanding of the field.

iii) Evaluating current security solutions against our thematic taxonomy based on their effectiveness and limitations.

iv) Identifying open research questions and challenges in this area, offering guidance for future studies.

The rest of the paper is organized as follows: Section II discusses the research methodology and its inclusion criteria. Section III introduces the thematic taxonomy of security challenges of containerization in shared kernel environments. Section IV discusses and analyzes state-of-the-art security practices for containerization. Section V provides a detailed review and comparative analysis of state-of-the-art containerization security solutions based on the thematic taxonomy defined in Section III. Section VI highlights ongoing research challenges and issues. Section VII concludes the paper with a summary of findings and recommendations for further research.

II. RESEARCH METHODOLOGY

This study adopts a rigorous, systematic approach to explore and analyze the security challenges associated with containerized environments, primarily focusing on kernel isolation

mechanisms in shared-kernel architectures. The methodology was carefully designed to address critical security concerns in containerization, ensuring that the findings provide meaningful insights into the evolving security landscape of cloud-native infrastructures. This section outlines the steps to gather relevant literature, select core security mechanisms, and evaluate modern solutions, culminating in a thematic taxonomy and comprehensive review of state-of-the-art container security practices.

A. RESEARCH OBJECTIVE

The primary goal of this research is to conduct an in-depth review of kernel isolation techniques used in containerized environments. The study aims to develop a thematic taxonomy highlighting the critical security challenges containers face when sharing an operating system (OS) kernel. It further conducts a comparative analysis of emerging and existing security frameworks. By addressing critical security issues related to containerization, this research provides actionable insights to enhance the security posture of shared-kernel containerized systems, making it a valuable resource for professionals in the field.

B. DATA COLLECTION PROCESS

This study follows a systematic approach to identify and analyze relevant research on container security. Searches were conducted using IEEE Xplore, ACM Digital Library, Elsevier, and Google Scholar with targeted keywords, including:

- "container security"
- "kernel isolation"
- "container escape attacks"
- "shared kernel vulnerabilities"

To ensure comprehensive coverage, the search was limited to papers published between 2017 and 2024, balancing foundational research with the latest advancements in the field.

C. INCLUSION AND EXCLUSION CRITERIA

The following criteria guided the selection of studies:

Inclusion Criteria:

- Studies focused on security mechanisms within containerized environments, emphasizing kernel-level isolation.
- Research addressing tools and frameworks for mitigating privilege escalation or container escape attacks.
- Papers presenting quantitative or comparative analyses of security frameworks.

Exclusion Criteria:

- Studies focusing exclusively on Virtual Machine (VM) security.
- Research is limited to container orchestration without addressing kernel-level isolation.
- Outdated studies (published before 2017) unless they introduce foundational concepts.

D. SELECTION OF SECURITY FRAMEWORKS AND MECHANISMS

The evaluation of security frameworks focused on their relevance to containerized environments with shared-kernel architectures. Particular attention was given to tools and frameworks recognized for mitigating privilege escalation attacks, container escape attacks, and securing kernel interactions. The selected mechanisms were assessed based on their effectiveness in:

- Enforcing isolation
- Managing resources
- Filtering system calls

Technologies such as seccomp, eBPF, and cgroups were prioritized, along with hardware-based solutions like Trusted Platform Module (TPM) and Software Guard Extensions (SGX), which play a crucial role in maintaining security at the kernel level.

III. TAXONOMY OF CONTAINERIZATION SECURITY

This section discusses the thematic taxonomy of containerization security in the cloud environment. Figure 1 highlights some of the essential features that help us understand the security challenges of containers and propose a better mechanism for finding attacks in the container ecosystem. Adopting containerization introduces security challenges related to the layered architecture of the container, encompassing the operating system kernel, container instances, and orchestration tools. Container security faces numerous challenges, including kernel vulnerabilities, meltdown, and spectre attacks, inter-container and host-container threats, insufficient isolation, software vulnerabilities, insider risks, and concerns regarding container image security, etc. [2], [34] This thematic taxonomy classifies container security on a set of parameters standard in most literature, including (A) Types of containers, (B) Security purpose, (C) Attack surfaces, (D) Threat actors, (E) Threats, (F) Security isolation, (G) Security Requirements.

A. TYPES OF CONTAINERS

Containerization has revolutionized how we build, release, and manage applications by encapsulating an application and all its dependencies into a single container. However, this approach comes with security concerns due to the complexity of the container architectural models. These challenges include kernel-level vulnerabilities that affect all containers, threats originating from inside and outside the containerized environment, and issues related to the security of the container images. Indeed, container security is a multi-layered domain that includes various types of containers, each with specific purposes. To enhance container security effectively, it is essential to distinguish between system containers and application containers and to understand the distinct security implications associated with each. Figure 2 shows the conceptual difference between system and application containers.

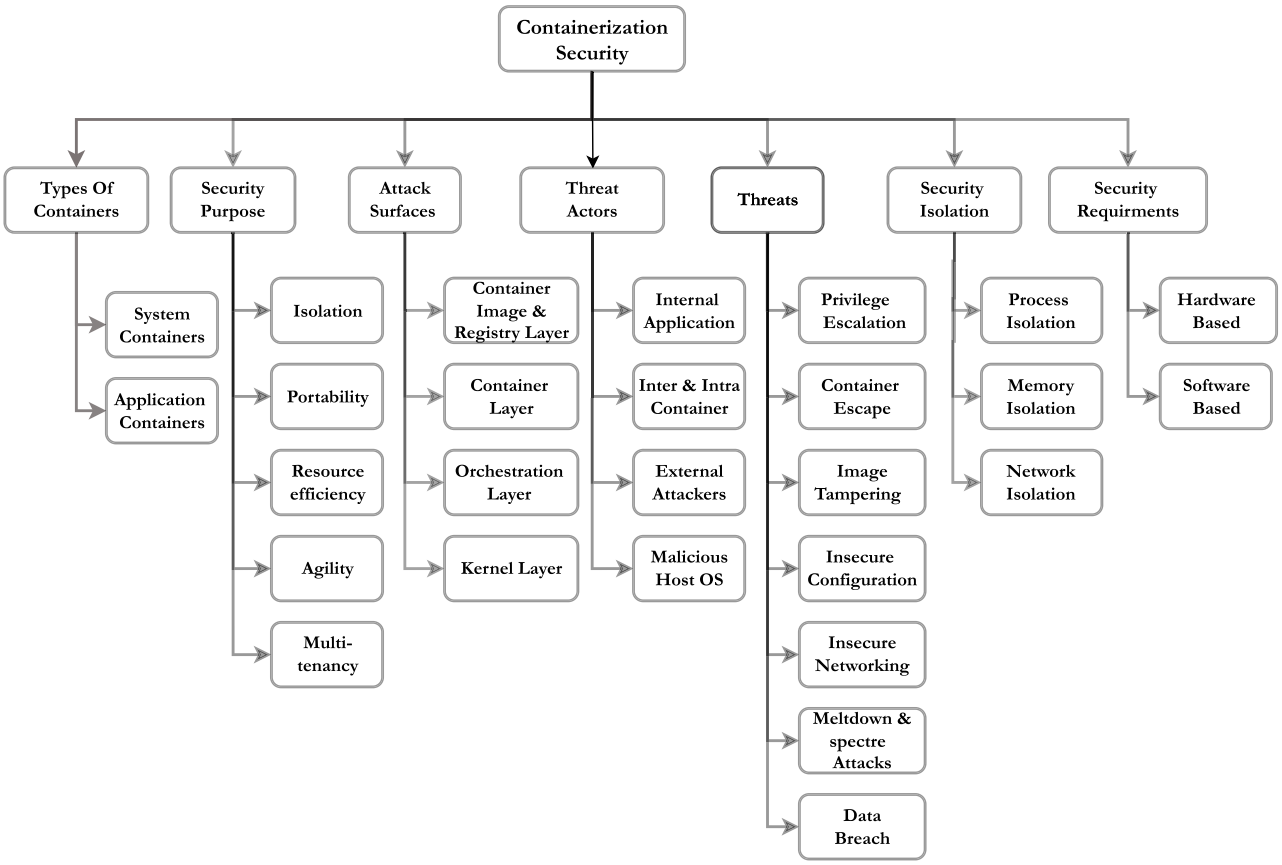

FIGURE 1. Thematic taxonomy of containerization security challenges.

1) SYSTEM CONTAINERS

System containers hold system-level services or utilities, not applications resembling VMs. They allow multiple processes to operate separately from one another while sharing the host system's kernel and providing isolation to user space. This setup streamlines installing and configuring different applications, libraries, and settings specific to the container, ensuring no container accesses irrelevant assets. System containers typically host traditional or monolithic applications, creating a consistent environment across multiple instances using templates or images. Examples of system containers include LXD (Linux Containers Daemon) and LXC (Linux Containers), with LXC built on top of LXD. LXD serves as system-level virtualization and orchestration software, offering a command-line interface for managing containers.OpenVZ is another open-source containerization system for Linux that provides isolated containers through system-level virtualization, each with its own file system, processes, and network stack. Linux-VServer facilitates multiple isolated virtual environments (Virtual Private Servers or VPS) on a single server.

2) APPLICATION CONTAINERS

A specific type of containerization technology ideally suited for bundling applications with their dependencies is commonly known as application containers. These containers are particularly beneficial for microservices architecture as they provide a stateless, scalable solution on demand without the need for coordination between pod orchestration systems and app hosting. Application containers encapsulate all runtime components, such as files, libraries, and environment variables, allowing applications to operate in an isolated environment separate from other applications. Thus, an application can run on the same underlying operating system kernel while maintaining isolation from other applications. These practices facilitate uniformity and agility across different environments, enabling the deployment and management of applications in various scenarios. Prominent technologies in application containerization include Docker, Containerd, rkt (also known as "rocket"), and Podman.

B. SECURITY PURPOSE

Within containerization, security serves several crucial roles by safeguarding containerized applications, the data they process, and the underlying infrastructure from unauthorized access or compromise. The primary security objectives include ensuring the confidentiality, integrity, and availability of applications and data and facilitating the execution of systems and functions in compliance with regulatory requirements and industry best practices.

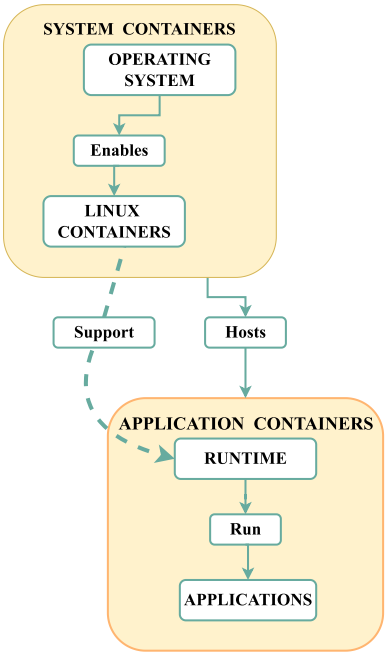

FIGURE 2. System container & application container.

1) ISOLATION

The primary security goal in container-based systems is isolation, which is essential for creating a secure environment that allows containers to operate safely. This process of isolation involves separating container processes and file systems from each other and from the host system to prevent the spread of malicious activities from one container to another or to the host system. Isolation effectively reduces the overall area of impact when a vulnerable element of the system is compromised, thereby minimizing the likelihood that one weak component could overcome the entire system or network.

2) PORTABILITY

Containers are designed to be portable across various environments, transitioning functionality from local developer workstations to production servers. Robust security mechanisms are crucial to prevent this portability from introducing vulnerabilities, such as those arising from insecure libraries or specific configurations that could expose confidential data. It is essential to continuously scan containers for vulnerabilities and ensure they are operated securely, regardless of location. This proactive approach to security helps maintain the integrity and confidentiality of containerized applications across different deployment settings.

3) RESOURCE EFFICIENCY

Containers operate under the ''less is more'' philosophy, making them ideal for running applications on a single host. However, while this efficiency in resource utilization optimizes performance, it can also expose vulnerabilities to attackers, leading to resource depletion or excessive resource consumption within or among containers on a shared server.

4) AGILITY

Containers provide flexibility, allowing developers to quickly develop, test, and launch their applications without worrying about infrastructure or dependencies. However, this flexibility also enables attackers to act rapidly by launching containers or exploiting vulnerabilities due to quick deployment speed.

5) MULTI-TENANCY

In multi-tenant environments, containers are typically used by multiple apps or users sharing access to resources. The interconnected nature of these containers presents a security risk, as it enables hackers to move ''laterally'' within the system. This allows them to target and potentially gain control of several applications simultaneously. Such environments require stringent security measures to prevent unauthorized access and protect all tenants effectively. Essentially, the focus on the security and objectives of containers aims to establish a stable and compliant application environment. Businesses can leverage the benefits of containerization and safeguard against risks by prioritizing isolation, portability, resource optimization, flexibility, and robust security measures.

C. ATTACK SURFACES

In containerization, attack surfaces refer to the layers and points in a containerized setting that can be targeted for security breaches and exploitation by actors. These vulnerable points exist across layers, each with its set of weaknesses and potential risks.

1) CONTAINER IMAGE & REGISTRY LAYER

At the Container Image & Registry Layer, the primary focus is on securing the container images and the registries that store these images. This includes vulnerability scanning of images, verifying image integrity and consistency, enforcing access controls on registries, and encrypting image data when it is transferred or stored. Furthermore, policies for image signing and verification provide additional quality assurance and enhance community confidence in the authenticity of the posted information.

Container images, especially those sourced from well-trusted registries, may still harbor vulnerabilities in their underlying operating systems, libraries, or application dependencies. Exploiting these weaknesses can lead to unauthorized system access, execution of malicious content, or privilege escalation within containers. Additionally, registries of container systems are susceptible to security risks. Without robust security measures, registries are at risk of unauthorized access or having malicious code injected into images by an intruder. Therefore, implementing stringent access controls, encryption protocols, and digital signatures is essential to prevent or mitigate these threats.

2) CONTAINER LAYER

The Container Layer is responsible for the security of the container's runtime environments. Containerization is a process where containers operate independently but share the host system's OS kernel. Container runtime may contain vulnerabilities that attackers could exploit to bypass container isolation controls and gain access to the host system. Such breaches could lead to data leakage, unauthorized access to websites, compromise of other containers on the same host, or alterations to the data within a container. Attackers can exploit this technology in various ways, such as exploiting kernel vulnerabilities, manipulating container configurations, escaping from the container's isolation, and executing arbitrary code on the host system. This underscores the importance of robust security measures to prevent container escape attempts and maintain the system's integrity.

3) ORCHESTRATION LAYER

The Orchestration Layer plays a crucial role in mounting and managing numerous containers. Security measures in this layer focus on strengthening the orchestration platform, ensuring secure communication between its components, and implementing stringent security policies for container scheduling and deployment. Common mechanisms such as authentication, authorization, and role-based access control are employed to manage access to orchestration resources and Application Programming Interfaces (APIs).

APIs security is critical in container orchestration platforms, as these APIs, which manage container-based applications, are exposed. Vulnerabilities such as inadequate authentication or authorization can allow attackers to gain unauthorized control over the orchestration environment, modify container loading, or access sensitive data. These vulnerabilities could be exploited by hackers to disrupt services, escalate permissions, or compromise the integrity of the applications running in containers. Ensuring correct configurations and adherence to security standards is vital for protecting the orchestration layer, impacting the overall system's stability and security.

4) KERNEL LAYER

The Kernel Layer is crucial for protecting the integrity of the host operating system and its kernel, which underpins the containers. This protection is achieved through several security enhancements, such as optimizing the kernel for security, promptly implementing security patches, and implementing Mandatory Access Control (MAC) mechanisms like AppArmor or SELinux to enforce strict access controls. Kernel runtime integrity monitoring, secure boot, and hardening of the kernel and system files help prevent unauthorized access and maintain the security of the entire container environment. Root vulnerabilities in the kernel pose a significant threat to container isolation and resource management, thereby risking the entire infrastructure. Malicious actors may exploit these vulnerabilities to penetrate the containerized infrastructure, potentially causing service failures, data breaches, and system compromises. Furthermore, the close interaction of container runtime with the host kernel can expose the kernel to attacks, allowing unauthorized access or control over containerized environments. To secure containerized ecosystems, a comprehensive approach that is flexible enough to cover all advised layers of protection is necessary. By mitigating these risks, organizations can build resilience and secure their containerized infrastructure against continually advancing threats.

D. THREAT ACTORS

Threat actors in containerization security are individuals, entities, or processes that pose a risk to the integrity, confidentiality, and availability of containerized environments. These actors exploit vulnerabilities or weaknesses within the container infrastructure for various malicious purposes. Understanding the different threat actors is crucial for effectively mitigating security risks and safeguarding systems and data.

1) INTERNAL APPLICATIONS

Internal applications refer to legitimate services, microservices, or processes deployed within containers in a containerized environment. While these applications are not inherently malicious, they can inadvertently introduce vulnerabilities due to insecure coding practices, outdated dependencies, or misconfigurations. Internal applications may inadvertently expose sensitive data, create security loopholes, or present attack surfaces exploitable by malicious actors. For example, a poorly configured internal application might inadvertently expose a database containing sensitive information.

2) INTER-CONTAINER & INTRA-CONTAINER THREATS

Inter-container threats involve interactions and communications among containers within the same or across multiple hosts, while intra-container threats pertain to vulnerabilities within individual containers. Inter-container threats may arise from insecure network configurations, weak access controls, or insufficient isolation between containers. Intra-container threats may stem from vulnerable application components or misconfigured permissions within containers. Inter-container threats may result in lateral movement, data leakage, or unauthorized access between containers. Intra-container threats can lead to container escape, privilege escalation, or compromise of containerized applications. For example, a container with elevated privileges may be vulnerable to exploitation by an attacker, leading to a breach of the entire containerized environment.

3) EXTERNAL ATTACKERS

External attackers are individuals or entities attempting to breach containerized environments from outside the system. External attackers target vulnerabilities in container images, runtimes, orchestrators, or exposed services to gain unauthorized access or execute malicious actions. They may

launch various attacks, such as container escape, network-based attacks, DoS attacks, or data exfiltration attempts, to compromise containerized applications. For example, an external attacker might exploit a known vulnerability in a container runtime to gain unauthorized access to the underlying host system.

4) MALICIOUS HOST OS

The host OS serves as the foundation for container runtime environments and provides essential resources and services. A compromised or malicious host OS can serve as a platform for attacks against containerized environments, enabling exploitation of kernel vulnerabilities or unauthorized access to containerized applications. A malicious host OS can facilitate container breakout, privilege escalation, or unauthorized access to sensitive data within containers. For example, a compromised host OS may allow an attacker to escalate privileges within a container and gain access to sensitive information or resources. By understanding the different threat actors, organizations can implement appropriate security measures to mitigate risks and protect containerized environments from various threats. This may include implementing robust access controls, regularly updating software components, and monitoring for suspicious activities.

E. THREATS

In the context of containerized environments, threats refer to potential malicious activities or vulnerabilities that could compromise the security, integrity, or availability of containers and their underlying infrastructure. These threats can range from internal vulnerabilities within the containerized applications to external attacks targeting the container ecosystem. Understanding these threats is crucial for implementing effective security measures and maintaining a robust defense posture.

1) PRIVILEGE ESCALATION

Privilege escalation occurs when an attacker gains unauthorized access to normally restricted resources or capabilities, potentially leading to full control over the container or host system. In containerized environments, this can happen if a container runs with more privileges than necessary or if there are vulnerabilities within the container or host OS that an attacker can exploit to gain elevated permissions.

2) CONTAINER ESCAPE ATTACKS

Container escape refers to the scenario where malicious code within a container breaches the isolation boundaries, allowing it to interact directly with the host OS or other containers. This can lead to unauthorized access to the host system, data leakage, or the ability to affect the operation of other containers. Container escapes are particularly concerning in shared environments, where they can lead to cross-container attacks.

3) IMAGE TAMPERING

Image tampering involves modifying container images to include malicious code or vulnerabilities. Attackers might push tampered images to public or private registries, tricking unsuspecting users into running compromised containers. This type of threat underscores the importance of image scanning, signing, and verification practices to ensure the integrity and security of container images.

4) INSECURE CONFIGURATION

Insecure configurations are common vulnerabilities in containerized environments. They can arise from overly permissive settings, default credentials, unnecessary open ports, or lack of security features. Attackers can exploit such configurations to gain unauthorized access, execute arbitrary code, or escalate privileges within the container or the host system.

5) INSECURE NETWORKING

Containers often communicate with each other and with external services, making the network a critical vector for attacks. Insecure networking practices, such as unencrypted traffic, lack of segmentation, or improper firewall rules, can expose containers to interception, unauthorized access, and network-based attacks.

6) MELTDOWN & SPECTRE ATTACKS

Meltdown and Spectre are hardware vulnerabilities that affect many modern processors, allowing attackers to read sensitive information from the system's memory, including passwords and encryption keys. Containers share the same underlying hardware and OS kernel, making them potentially vulnerable to such side-channel attacks if the host system is not patched.

7) DATA BREACH

A data breach in containerization occurs when unauthorized access leads to the exposure of sensitive information stored within or accessible by containers. This can result from various factors, including insecure APIs, vulnerabilities within the containerized application, or compromised container images. Data breaches can have significant legal, financial, and reputational consequences for organizations.

Addressing these threats requires a multifaceted approach, including implementing best practices for container security, regular vulnerability scanning, network segmentation, applying the principle of least privilege, and staying updated with the latest security patches and updates. By understanding and mitigating these threats, organizations can enhance the security of their containerized environments and protect their critical assets.

F. SECURITY ISOLATION

Security isolation is a cornerstone of container security, crucial for maintaining the integrity and confidentiality of containerized applications running in shared environments.

It involves segregating and restricting the execution context of containers to prevent unauthorized access or leakage of resources among them. This segregation is particularly important in a shared kernel architecture, where containers share the same underlying OS's kernel but need to operate as if they are on separate machines. Some important isolation types in the context of security are discussed below

1) PROCESS ISOLATION

Process isolation ensures that processes running within a container are isolated from processes running in other containers or on the host system. This is achieved through namespaces in Linux, which provide each container with a unique view of the system, including its own process tree, network interfaces, user IDs, and mounted file systems. Process isolation prevents malicious processes within a container from affecting other containers or the host, thereby limiting the scope of potential attacks.

2) MEMORY ISOLATION

Memory isolation restricts containers from accessing memory allocated to other containers or the host system. This is primarily enforced through cgroups (Control Groups) in Linux, which limit the amount of memory a container can use. Additionally, memory isolation mechanisms ensure that memory pages are not shared between containers unless explicitly allowed, preventing sensitive information from being leaked or tampered with across container boundaries.

3) NETWORK ISOLATION

Network isolation separates the network stack for each container, ensuring that containers have their own IP addresses, routing tables, and port spaces. Namespaces facilitate this, allowing containers to communicate with each other and the outside world in a controlled manner. Network policies can further restrict the flow of network traffic between containers, preventing unauthorized access and mitigating network-based attacks. In a shared kernel architecture, the kernel is the only component not abstracted into containers, which introduces potential risks if a container manages to bypass isolation mechanisms. Security isolation plays a vital role in fortifying container security in such environments. By implementing robust security isolation mechanisms, organizations can leverage the efficiency and flexibility of containerized environments while maintaining a strong security posture in shared kernel architectures. This isolation is crucial not only for protecting individual containers but also for ensuring the overall resilience of the container ecosystem against various security threats.

G. SECURITY SOLUTIONS

Security solutions for containerized environments are designed to protect containers, their applications, and the infrastructure they run on from various threats and vulnerabilities. These solutions can be broadly categorized into hardware-based and software-based solutions, each offering distinct mechanisms for enhancing the security posture of container deployments.

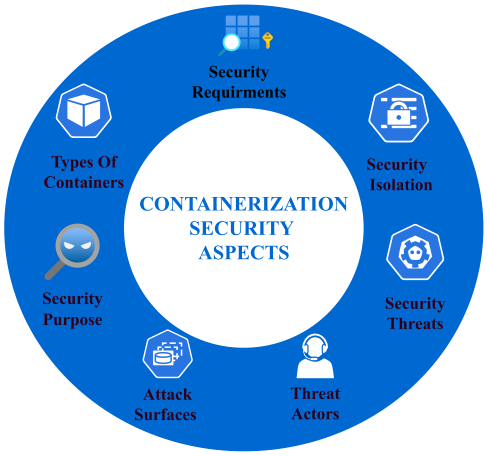

FIGURE 3. Containerization security aspects.

1) HARDWARE-BASED SECURITY SOLUTIONS

Hardware-based security solutions utilize hardware's physical features to implement security measures inherently resistant to tampering and bypassing. These solutions provide a robust foundation for securing containerized environments by ensuring the integrity and confidentiality of data directly at the hardware level. Among the critical use cases of hardware security modules, the Trusted Platform Module (TPM) is pivotal in securing containers within a shared kernel environment. TPM is utilized to securely store encryption keys and credentials, ensuring their protection even in the event of system compromise. Hardware Security Modules (HSMs), which manage and store cryptographic keys, play a crucial role in encrypting container data to reinforce communication security. Intel Software Guard Extensions (SGX) offer protected execution environments known as enclaves for handling sensitive container workloads. SGX ensures data encryption in memory and minimizes exposure risks.

2) SOFTWARE-BASED SECURITY SOLUTIONS

Software-based security solutions rely on software to provide security controls and measures. These solutions are more flexible and can be easily integrated into the container lifecycle and CI/CD pipelines, offering comprehensive protection against various threats. Some of the use cases of software-based security solutions are: Container Security Platforms: These platforms are designed to be proactive, providing comprehensive security features such as vulnerability scanning, runtime monitoring, and compliance enforcement. They directly integrate with CI/CD pipelines, enabling preemptive threat mitigation and giving you peace of mind about your container security.

Access Control and Identity Management: These tools efficiently manage access to containerized applications, implementing single sign-on (SSO) measures to streamline

secure user access. Their effectiveness will give you confidence in the security of your containerized applications. Encryption and Data Protection: The software mechanisms in place encrypt data at rest and in transit, providing robust protection for sensitive information. This robustness will make you feel secure about the confidentiality of your data. Firewalls and Network Policies: Enforce container traffic rules, facilitating network segmentation and reducing the risk of internal spread during an attack.

By combining hardware-based and software-based security solutions, organizations can create a layered security approach that addresses the diverse threats facing containerized environments. This holistic strategy ensures the confidentiality, integrity, and availability of containerized applications and their data, aligning with best practices for container security.

Figure 3 illustrates key security aspects necessary for ensuring strong container isolation in a shared kernel architecture. To achieve robust security, it is essential to identify the types of containers in use and their specific security purposes. Furthermore, recognizing potential attack surfaces and the threat actors targeting the containers is crucial. Understanding the various threats within a shared kernel environment helps determine the necessary isolation techniques. Finally, evaluating the security requirements, whether software or hardware-based, is essential to providing optimal protection. By addressing these elements, developers and organizations can enhance container security in shared kernel systems

IV. REVIEW AND COMPARATIVE ANALYSIS OF STATE-OF-ART CONTAINERIZATION SECURITY SOLUTIONS

This section consists of two parts. The first part offers a detailed review of the state-of-the-art containerization security solutions, highlighting the most recently published research findings. The second part provides a comparative analysis of these solutions based on the thematic taxonomy defined in section III.

A. STATE-OF-THE-ART CONTAINERIZATION SECURITY SOLUTIONS

This section comprehensively reviews state-of-the-art containerization security solutions, emphasizing the most recent research. We initiate with a succinct overview of the existing solutions in Table 1, followed by an in-depth exploration of each study. This structure provides a robust understanding of the security landscape, laying a solid foundation for the subsequent detailed analysis of individual solutions.

The review of state-of-the-art containerization security solutions is based on a comprehensive set of criteria. The selection spans two-time frames to ensure a thorough understanding of recent advancements and foundational research. The works reviewed cover the period from 2017 to 2023, focusing on studies published between 2020 and 2024. This time frame captures the dynamic evolution of security techniques in containerized environments, keeping the audience updated on the latest advancements.

The distribution of selected works for comparative analysis is as follows: 4% from 2024, representing the latest advancements in container security. 38% from 2023, reflecting significant research activity from the past year. 35% from 2022, which includes foundational solutions that continue to influence current practices. 23% from 2021 and 8% from 2020, addressing earlier developments in kernel security and container escape prevention mechanisms. The reviewed literature includes various sources, such as technical papers and frameworks, offering a comprehensive view of container security approaches. Including diverse studies ensures a balanced discussion of theoretical advancements and practical implementations in the field.

1) sysE

The authors [35] introduce a technique called sysE, which looks at attack data from various angles and spots similarities in system calls across different vulnerabilities over time. This data is then organized and grouped to create important features that can be learned by TinyML-based detectors, boosting their effectiveness. These detectors become adept at identifying new attacks as they emerge without needing constant and expensive retraining. The method not only slows down the aging process of the models but also cuts down on the effort required for labeling data. This makes it easier to update models in networks with limited resources, such as UAV networks. Moreover, it lowers maintenance costs, making it more feasible to deploy TinyML-based models across different platforms, improving overall efficiency and cost-effectiveness. By broadening the representation space and understanding of attack data, this method improves the detection capabilities of container escape detectors. Additionally, it establishes a standardized way of collecting and storing attack data, enabling real-time acquisition and facilitating analysis. However, while it's shown promise in experiments with evolving datasets, questions remain about its performance on different datasets and in real-world situations. Moreover, its effectiveness in detecting entirely new and unknown attacks isn't fully clear yet. Additionally, while it reduces maintenance costs, its impact on overall model performance and security warrants further investigation.

2) NIMOS

The paper [36] investigates the effectiveness of using system call sequence patterns as a defense mechanism for container security, focusing on preventing container escape scenarios. Researchers utilized NIMOS, an automated analysis tool, to examine real-world exploit codes. They discovered a wide array of common system call sequences within these exploits, varying in length and composition. Their findings demonstrated that the sequence-based method effectively blocked a considerable portion of exploits that were not stopped by default Docker's seccomp settings. Even specialized seccomp

TABLE 1. State-of-the-art container security solutions in shared kernel architecture.

Existing Solutions	Security Purpose	Attack Surface	Threat Actors	Threats Addressed
sYsE [35]	Isolation, Agility, Multi-tenancy	Image, Registry	External Hackers	Container Escape, Privilege Escalation
NIMOS [36]	Multi-tenancy	Container	Malicious Users	Container Escape
μSWITCH [37]	Isolation, Resource Efficiency	Kernel	Insider Threats	Unauthorized Access
Confine [38]	Isolation, Resource Efficiency	Kernel	External Attackers	Privilege Escalation, System Call Filtering
SysVerify [39]	Isolation, Portability	Kernel	Internal and External Threats	Malicious Activity
KRSIE [40]	Isolation, Portability	Kernel, Orchestration	Insider Threats	Access Control Violations
SandBPF [41]	Isolation, Portability, Resource Efficiency, Agility	Image, Registry, Orchestration	Both Internal and External	CVE Exploits, Kernel Attacks
SySched [42]	Isolation	Kernel	External Threats	System Call Filtering
Seccomp-eBPF [43]	Isolation, Agility	Kernel	Insider Threats	System Call Restrictions
CODA [44]	Isolation	Kernel, Application	Insider Threats	Denial-of-Service Attacks
Bastion+ [45]	Isolation	Network	External Threats	Network Security Enforcement
CIMM [46]	Resource Efficiency	Image, Registry	Insider Threats	Image Tampering
PACED [47]	Isolation	Kernel, Orchestration	Insider Threats	Container Escape Detection
FIT framework [48]	Isolation, Resource Efficiency	Kernel	Insider Threats	Fault Recovery
PBRIEM [49]	Isolation	Kernel	Insider Threats	Process Monitoring
ContainerGuard [50]	Isolation	Kernel	Insider Threats	Spectre, Meltdown
PPASes [51]	Isolation, Resource Efficiency	Kernel	Insider Threats	Memory Protection
TCX [52]	Isolation, Portability	Kernel, Hypervisor	Insider and External Threats	Trusted Execution
HSCPA [53]	Isolation	Kernel	Insider Threats	System Call Profiling
GYroidOS [54]	Isolation, Agility	Kernel	Insider Threats	Kernel Hardening
CCSPS [55]	Resource Efficiency	Orchestration	Insider Threats	Container Escape Prevention
Sprofiler [56]	Isolation	Kernel	External Threats	System Call Filtering
Prof-gen [57]	Isolation	Kernel	Insider Threats	Static Analysis
saBPF [16]	Isolation, Portability	Kernel	Insider Threats	System Call Auditing
Lic-Sec [58]	Isolation, Agility	Kernel	Insider Threats	Mandatory Access Control
TZ-Container [59]	Isolation	Kernel	Insider Threats	Trusted Zone Isolation
BPFCONTAIN [60]	Isolation, Agility	Kernel, Orchestration	External Threats	eBPF-based Monitoring
bpfbox [61]	Isolation, Agility	Kernel	Insider Threats	Process Confinement
NNASC [62]	Isolation, Resource Efficiency	Kernel	Insider Threats	Anomaly Detection

profiles failed to defend against some attacks, while the sequence-based approach mitigated a significant majority of them. This suggests that prioritizing sequence-based defense mechanisms could significantly reinforce container security. The study concludes by advocating for further efforts to implement and refine this approach, highlighting its potential to enhance the security of container runtimes. Despite its success, the paper lacks discussion on critical implementation aspects such as scalability, space consumption, and potential kernel overhead. It also overlooks the limitations of the NIMOS tool and fails to address runtime performance impacts. Furthermore, the paper does not delve into defense against syscall manipulation or evaluate runtime overhead, suggesting the need for further research and refinement before practical deployment.

3) μSWITCH

Peng et al. [37] proposed a hardware-based solution for in-process memory isolation referred to as μSWITCH. μSWITCH isolates process memory and kernel resources using the Memory Protection Keys (MPK) based memory separation technique. Through this approach, the authors overcome process isolation issues and minimize performance overhead. μSWITCH provides a secure mechanism for kernel context switching for memory structures shared between user space and the kernel while also controlling unnecessary context switching of implicit system calls by isolating resources of individual processes and allowing context switching of necessary system calls. This method not only improves process isolation but also minimizes performance overhead. However, there are some limitations, this method provides isolation for different applications and also protects code from vulnerabilities but lacks network and container isolation mechanisms. This technique depends on Intel MPK hardware, which is not a generalized solution but rather a hardware-based one. Additionally, secure isolation of both memory and the kernel is a complex task. Challenges such as incomplete kernel resource isolation, temporary abstraction

of kernel context, and filtering of implicit system calls are some limitations of the μSWITCH method.

4) CONFINE

The Confine system [38] makes container environments more secure by carefully controlling the system calls that each container can make. Confine looks closely at the program running inside a container and figures out all the system calls it needs to work properly. Once Confine knows what system calls are necessary, it creates a set of rules (called a Seccomp filter) that only allows those specific system calls and blocks everything else. Confine uses two layers of control; first, it creates a general rule that applies to all programs in the container. Second, it makes a specific rule that removes certain system calls that are only needed when the container is starting up. Confine pays special attention to the details of the system calls, considering not only which ones are used but also the argument values that provide more fine-grain system call filtering. Confine implements these rules when the container starts running. This ensures that only the necessary system calls are allowed, reducing the chances of a security breach and helping protect the container and the overall system from potential security threats. The proposed solution, while promising, has its limitations. Dynamic analysis faces challenges in capturing all the code required for future workload scenarios. This constraint implies a potential gap in comprehensively assessing system call behaviors, particularly in evolving environments where new functionalities may be introduced over time. Moreover, the limited number of applications and libraries directly utilizing system calls could improve its effectiveness. Consequently, the scope for identifying argument values associated with system calls is restricted, impacting the solution's ability to create fine-grained policies tailored to specific application needs. These shortcomings underscore the need for further research and development to address workload environments' dynamic nature and enhance the solution's capability to capture diverse system call behaviors and argument values.

5) SysVERIFY

Zhan et al. [39] proposed a security solution for containers known as Sysverify. SysVerify is a Systematic Dependent Syscall Analysis Approach, it reduces the attack surface of the Linux kernel by filtering the accessible system calls of running containers. It consists of two main parts: static analysis and dynamic verification. In the static analysis phase, Sysverify examines the program's code to understand which system calls it relies on. This involves analyzing both the binaries and the source code of dependent libraries. Sysverify then creates a list that contains the program's system calls with the specific commands they use to communicate with the kernel. It also analyzes indirect system calls to identify their destination and relationship to other system calls, providing a more detailed way to reduce potential attacks. In the dynamic verification phase, Sysverify utilizes a Linux security feature called Seccomp to filter the system calls of the target program. It actively blocks indirect or rarely invoked system calls, essentially creating boundaries around what the program can do. Sysverify examines how a program communicates with the Linux kernel, develops a strategy to limit those system calls, and tests to ensure that the limitations are effective.SysVerify is also subject to several notable limitations. Firstly, its support is currently confined solely to Docker containers, thus excluding other container types. This limitation restricts its applicability in environments where diverse container technologies are employed. Moreover, the solution's reliance on static analysis. Static analysis techniques may inadvertently produce false positives, which could inadvertently expand the attack surface rather than mitigate it. Furthermore, SysVerify's efficacy may be compromised when confronted with complex scenarios, such as indirect calls, where static analysis struggles to assess security risks accurately. These limitations underscore the necessity for ongoing refinement and expansion of SysVerify's capabilities to address a broader range of container types and to enhance its accuracy and reliability in detecting security threats.

6) KRSIE

The proposed approach, Kubernetes Runtime Instrumentation and Enforcement System (KRSIE) [40], relies on eBPF technology to enforce security policies dynamically within Kubernetes environments. Acting as a Linux Security Module (LSM), KRSIE permits users to attach modular BPF programs to various security hooks, granting them the flexibility to define their own Mandatory Access Control (MAC) policies with custom code. To utilize KRSIE, users need to code a BPF program with LSM hooks and execute it in the kernel. This program, employing LSM probe, operates just before policy enforcement, adjusting conditions and executing actions based on parsed information. The effectiveness of KRSIE is showcased through real-world scenarios, demonstrating its capacity to enhance container security in cloud-native setups by controlling behaviors at runtime. However, the method has certain limitations. It lacks a comprehensive comparison of KRSIE with other LSM-based solutions, like AppArmor and SELinux, in terms of performance, effectiveness, and ease of implementation. Furthermore, the evaluation is based on a single real-world scenario, possibly not covering the full spectrum of container runtime security challenges. The paper also overlooks discussing potential drawbacks or limitations of using eBPF technology in Kubernetes environments. Additionally, there's a lack of analysis regarding the performance impact and overhead introduced by KRSIE during container runtime operations. Lastly, insights into the scalability of KRSIE and its ability to handle numerous containers and security policies in production environments are missing.

7) SandBPF

SandBPF [41] is a Software-based kernel isolation technique that dynamically sandboxes eBPF programs, enabling unprivileged users to extend the kernel safely. It utilizes software fault isolation (SFI), transforming memory access and control-transfer instructions to confine programs within designated memory regions. SandBPF ensures memory safety and control-flow integrity by employing address masking and trampolined control transfers. Address masking restricts program access to its designated memory space, while trampolined control transfers ensure program calls only permitted entry points when venturing outside its domain. These measures confine eBPF programs within their sandbox, enhancing kernel extension safety. However, SandBPF incurs a constant overhead in sandbox management, impacting system performance. Web server benchmarks introduce an overhead of 0%-10%, potentially affecting system efficiency. While it addresses exploits overlooked by eBPF's native safety mechanisms, it may not cover all vulnerabilities. The reliance on SFI techniques poses its limitations and potential vulnerabilities. Further evaluation is needed to assess SandBPF's effectiveness and scalability in real-world scenarios and large-scale systems. Additionally, detailed implementation considerations, such as compatibility with different kernel versions and potential conflicts with other extensions, require further exploration.

8) SySched

SySched [42] is a system call-aware container scheduler that minimizes a container's exposure to vulnerable system calls. It introduces a metric called Extraneous System call Exposure (ExS) to quantify the risk of container escape attacks. By reducing a container's exposure to extraneous system calls, SySched aims to mitigate the impact of compromised containers on the host kernel and other containers. SySched can operate dynamically at runtime or statically, adapting to changing workload behaviors or relying on pre-generated system call profiles. While it doesn't eliminate system call-based compromises, SySched provides a mechanism to contain and limit their impact, improving overall container security. However, despite its benefits, SySched has limitations. Firstly, it may not fully account for vulnerabilities based on the arguments of system calls. Secondly, it assumes single-container pods, which may need to align with the realities of multi-container environments. Lastly, obtaining accurate system call profiles can be challenging, particularly in dynamic environments with diverse workloads. While SySched represents a significant advancement in container security, it's important to acknowledge these limitations. Future research may address these challenges to further enhance SySched's effectiveness in securing container-based clouds against escape attacks.

9) Seccomp-eBPF

The paper [43] introduces a novel approach to system call filtering using eBPF, enhancing security policies and reducing early execution phase vulnerabilities. It creates a new Seccomp-eBPF program type (seccomp is a security computing module of Linux), enabling advanced filtering capabilities like statefulness and safe user memory access. This system integrates seamlessly with existing kernel privilege mechanisms, allowing unprivileged users to implement sophisticated filters. However, the paper needs to discuss the limitations of the existing Seccomp-cBPF method, particularly its inability to support stateful policies and the requirement for kernel modifications. The evaluation focuses on specific use cases, potentially overlooking other scenarios, and doesn't compare performance impacts with alternative techniques. Additionally, the paper doesn't delve into the learning curve or complexities associated with eBPF nor addresses potential security risks arising from its use. Lastly, compatibility issues with kernel versions and possible conflicts with other security mechanisms are not explored.

10) CODA

CODA [44] is a framework designed to detect CPU-exhaustion Denial-of-Service (DoS) attacks at the application layer within containers. It monitors CPU usage per connection and utilizes statistical techniques for attack detection. By tracing system calls via Linux eBPF at the host level, CODA tracks connection establishment and closure, marking the beginning and end of request processing for monitoring CPU usage. The framework can distinguish between legitimate and attack connections based on statistical differences in CPU consumption, accommodating programs in various languages and remaining transparent to the container. However, the proposed method is still being determined about its effectiveness across all container types and environments. Additionally, there's no mention of potential false positives or negatives, scalability in large-scale deployments, performance impact in production settings, or its efficacy against sophisticated attack techniques. Furthermore, there needs to be a comparison with existing detection methods to evaluate CODA's superiority. These limitations highlight areas for future research and enhancement of the CODA framework.

11) BASTION+ METHOD

Nam et al. [45] proposed a Bastion+ tool for container attack detection.Bastion+ provides secure isolation by implementing a network security enforcement stack, which controls container communication and keeps them safe. Bastion+ sets up a security system for each container, such as having a personal guard. This system ensures that each container only does what it needs to do on the network, following the rules for security and privacy. It creates a direct and secure pathway between containers. Imagine it as a secret route only the sender and receiver know, ensuring no one else can eavesdrop on what they're saying. This way, containers can talk to each other without others on the same system knowing what's happening.Bastion+ allows us to attach special security checks to the communication

between containers. These checks are like inspectors making sure everything is safe. It lets us decide what checks to use and in what order, adding an extra layer of security to ensure everything harmful gets through.Bastion+ acts as a personal security detail for each container, ensuring they communicate safely and securely, reducing the chances of anything terrible happening between them.Bastion+ exhibits certain limitations that warrant consideration in the context of container environments. Firstly, they lack dedicated security mechanisms to counter L3/L4 spoofing attacks, potentially leaving network infrastructures vulnerable. Furthermore, security policies explicitly tailored for network-privileged containers still need to be addressed, leaving a gap in comprehensive network security strategies. Additionally, the inability to effectively restrict network flow control due to the dynamic nature of IP addresses further complicates security enforcement efforts. These limitations underscore the need for continuous refinement and innovation in security solutions to effectively mitigate emerging threats and ensure comprehensive protection in containerized environments.

12) CIMM

Yang et al. [46] introduced the Container Integrity Measurement Module (CIMM) as an image security detection method. The CIMM method is utilized to identify malicious container images through an image measurement technique. The proposed method initially utilizes the Clair tool to obtain image layer information, including system and software package details and scans the mirror image for vulnerabilities. If the image is considered secure, it is downloaded successfully; otherwise, it is blocked. In the image security detection process, the CIMM module calculates image measurements. CIMM measures the code segment, data segment, and shared library information of the container, storing it in the database using a hash function. When the container is started, the CIMM module again measures the image and compares it with the stored measurements. If the image measurements are the same, the container can run successfully; otherwise, the image security system blocks it, reducing image tampering security issues. This method also handles process isolation and network isolation. Process isolation provides a access list of process commands that can be executed when the container runs, while network isolation is implemented through iptables. Overall, this method effectively reduces image tampering security issues. However, it still has some limitations. For instance, it may not efficiently handle security issues as image size increases, there might be computation overhead, and the process and network isolation techniques may be suitable only for small and limited image resources. Additionally, depending on a static list may create problems in handling dynamic complex runtime scenarios.

13) PACED

Abbas et al. [47] proposed a method PACED, which stands for Provenance-based Automated Container Escape Detection, which is a system designed to detect container escape attacks in microservices. It does this by looking at records of everything a system does during an attack. PACED gathers logs that detail all the actions a harmful program takes on a system. Examining these logs, it spots events where data moves between containers or a container and the host system. These particular events can signal an attempt to break out of a container.PACED uses a wise rule called ''privileged flow'' to find abnormal events in Docker and Kubernetes setups, pointing out any suspicious entities trying to surpass container boundaries. The system is accurate, almost perfect, and doesn't miss any real threats. Importantly, PACED doesn't just look at past data; it actively watches for and warns about attacks as they happen.PACED, while offering promising advancements in container security, has its limitations. Firstly, the fragility of container isolation within the PACED framework can leave applications vulnerable to attacks. The absence of systematic methods for vulnerability scanning presents a significant challenge, as it hinders proactive identification and mitigation of security weaknesses. Another concern lies in the incomplete Linux kernel namespaces utilized by PACED, which raises security and privacy apprehension. Lastly, the absence of systematic vulnerability scanning methods contributes to the tediousness of detection efforts, highlighting the need for more efficient and comprehensive scanning techniques. Addressing these limitations will further refine the PACED framework to reinforce container security and resilience against emerging threats.

14) FIT FRAMEWORK

The FIT framework proposed [48] in the paper aims to enhance security in containerized environments by automatically detecting and recovering from errors caused by malicious containers or accidental faults. It focuses on countering container escape attacks that aim to bypass security measures. The methodology involves a module for detecting errors based on specified security states and a recovery module to restore the system to a secure state. However, there are limitations to consider: the scalability of the verification approach may pose challenges, especially with frequent events; the dynamic nature of container environments may complicate the process; human intervention may still be required for identifying anomalies; and the current solutions lack feedback for automated recovery and root cause analysis. Although ongoing activities involve further development, the proposed methodology is still in the early stages and requires more investigation for feasibility.

15) PBRIEM

The paper [49] proposes a Process-based resource isolation and escape monitoring scheme (PBRIEM) for enhancing container security by studying container escape characteristics and utilizing a process-based approach for resource isolation and escape monitoring. By ensuring consistency between container and host process namespaces, abnormal processes

and container escapes can be identified. Access controls are employed to isolate resources, preventing unauthorized access to files. Additionally, a security monitoring system is implemented to monitor resource usage and container processes, alerting users to abnormalities. Experimental validation confirms the scheme's effectiveness in detecting and preventing container escapes, ensuring the reliability and security of containerized environments. However, the paper lacks detailed implementation specifics and technical mechanisms, making it challenging to evaluate effectiveness and feasibility. Performance and resource utilization impacts and scalability and compatibility with various container platforms and operating systems are not discussed. Furthermore, the referenced research by Thanh does not offer an actual implementation scheme, limiting available research on this topic.

16) ContainerGuard

ContainerGuard [50] is a system designed to detect meltdown and spectre attacks on big data platforms, particularly those utilizing container-based environments. It monitors the activity of multithreaded processes within running containers, collecting time-series hardware and software performance events. The proposed methodology uses exponential smoothing to handle missing values in the time-series performance event data while preserving the data's baseline shape. This smoothed data serves as input for the anomaly detector, aiding in accurate anomaly detection.ContainerGuard has several potential limitations. Firstly, the absence of clear performance metrics used for evaluation may hinder a comprehensive assessment of its performance. Moreover, ContainerGuard's limited information on addressing sophisticated attack techniques raises concerns about its ability to mitigate advanced threats effectively. Furthermore, the scope of evaluation may need to adequately capture real-world performance implications, potentially limiting its applicability in practical settings. Its dependency on accurate representations of normal behavior for detection and the absence of validation against real-world attack scenarios further underscore uncertainties about its practical efficacy.

17) PPASes

Introducing BlackBox [51], a container architecture designed to safeguard application data confidentiality and integrity without relying on the operating system's trust. It achieves this by implementing a container security monitor (CSM) that creates protected physical address spaces (PPASes) for each container, ensuring isolation from the underlying OS. By leveraging ARM hardware virtualization support and nested paging, BlackBox enforces PPASes, preventing direct information flow between containers and the OS. The CSM operates in ARM's hypervisor mode (EL2), configuring Stage 2 page tables and the System Memory Management Unit (SMMU). This setup ensures container CPU and memory state isolation by alternating between container and OS Nested Page Tables (NPTs) before and after transitioning to the OS.While providing superior security compared to traditional architectures, BlackBox may introduce complexities related to the use of encryption for sensitive data and has limitations such as the lack of support for writable memory-mapped file I/O and overhead in certain operations like fork and exec measurements. Additionally, its threat model primarily addresses OS vulnerabilities, leaving potential gaps in addressing availability attacks by a compromised OS. Further research on BlackBox could focus on improving performance, extending support for additional system calls, overcoming existing limitations, strengthening security measures, and assessing scalability and compatibility.

18) TCX

Trusted Container Extensions (TCX) [52] introduces a container security framework that merges the flexibility of standard containers with the robust security of hardware-backed Trusted Execution Environments (TEEs), like AMD SEV. This setup ensures the confidentiality and integrity of container workloads, even in untrusted cloud environments. Built on the Kata Containers project, TCX facilitates secure communication between containers, regardless of their execution location. Performance testing reveals minimal impact, with only a 5.77% overhead observed on the SPEC2017 benchmark suite. However, the paper overlooks exploring TCX's implementation with alternative TEE architectures and lacks a detailed analysis of its scalability, especially in dynamic environments. Moreover, the security evaluation primarily focuses on certificate validation, neglecting other potential vulnerabilities. Additionally, practical deployment challenges and integration with various cloud environments or container orchestration systems aren't thoroughly discussed. Despite its innovative approach, the paper could benefit from further exploration of these aspects for a more comprehensive understanding of TCX's applicability and effectiveness.

19) HSCPA

In Hybrid System Call Profiling Approach(HSCPA) [53] begins by creating an initial fine-grained access list through dynamic tracking, monitoring the actual system calls the container uses during its execution. Simultaneously, static analysis is conducted on the container and its dependencies to extract an additional access list, covering all necessary functionalities, even those not immediately evident during dynamic tracking. The approach analyzes the container's behavior in three phases: startup, runtime, and shutdown, dynamically enforcing fine-grained access lists based on the container's behavior and adjusting them as the container progresses through its lifecycle. When a process inside the container attempts a system call not present in the access lists, the approach may either terminate the process or log the system call for further investigation.HSCPA creates a dynamic access list by monitoring the actual system calls the container makes during its execution phases—startup, runtime, and shutdown. Concurrently, it extracts an additional access list from static

analysis of the container image, ensuring comprehensive coverage of necessary system calls. When a process within the container initiates a system call, the approach checks its presence in the dynamic access list. If absent, it consults the static access list. If the system call is not found in either list, the approach decides whether to terminate the container or log the system call for subsequent analysis. By monitoring and allowing only necessary system calls during different stages, HSCPA reduces security risks and attacks. However, HSCPA has several limitations. The lack of benchmarking tools complicates the validation process and hinders accurate assessment of HSCPA's effectiveness. Additionally, the dynamic nature of containerized environments makes it difficult to maintain a complete system call list, as dynamic analysis techniques may not capture all relevant system calls, leading to potential gaps in security coverage. This issue is further compounded by the reliance on a stateless access list model, which introduces the risk of mimicry attacks, where malicious actors may exploit vulnerabilities by imitating legitimate system calls. Addressing these limitations is crucial for refining HSCPA to ensure more accurate and comprehensive system call profiling, thereby enhancing container security in dynamic computing environments.

20) GyroidOS

The paper [54] proposed a solution for GyroidOS. Packaging Linux with a minimal surface introduces a novel system architecture called GyroidOS, designed to enhance the security of Cyber-Physical Systems (CPS) by reducing the attack surface of the Linux kernel within container environments. GyroidOS achieves this by limiting the accessible kernel functions from within containers, aiming to minimize potential vulnerabilities while maintaining compatibility with existing applications without requiring extensive modifications. Thecriticaly innovation of GyroidOS lies in its use of syscall emulation, which remaps redundant syscalls to others providing similar functionalities, thus containing unnecessary functionality. This approach ensures that user space functionality remains intact while reducing the potential attack vectors. GyroidOS employs two main methods for reducing the kernel attack surface: minimal syscall hooking and Seccomp filters. These techniques intercept and restrict syscalls that could be used maliciously, ensuring tighter security without compromising performance. However, GyroidOS does have limitations. The manual effort required to identify and implement syscall handlers may pose challenges, and while the architecture reduces the kernel attack surface, it may only eliminate some security threats. Additionally, while GyroidOS incurs minimal performance penalties, users should consider potential impacts in highly time-sensitive or performance-critical environments.

21) CCSPS

The CCSPS (Comprehensive Container Security Protection Scheme) proposed [55] a scheme against container escape related to container management procedures, which delves into the security challenges posed by container escapes within Docker and similar container management systems. To address this threat, the authors propose a multifaceted defense approach consisting of two phases: detection of attack patterns and a holistic protection scheme. In Detection of Attack Patterns, the method involves identifying patterns of container escape attacks and pinpointing weaknesses in container management programs. In the Holistic Protection Scheme, a comprehensive defense strategy aims to mitigate container escape risks by addressing vulnerabilities within container management systems. The proposed solution focuses on reinforcing the security of container management procedures. It includes implementing security monitoring systems, enhancing resource isolation, patching vulnerabilities, enforcing policies, and rigorous testing. These measures collectively aim to fortify the integrity of containerized environments and the applications they support. However, some limitations exist in the proposed defense mechanism. These could encompass performance impacts, complexity in implementation, adaptation to evolving threats, challenges in achieving complete isolation, and potential gaps in defending against insider threats.

22) SPROFILER

The paper [56] aims to present Sprofiler, a tool designed to automatically generate security policies for system call filters using Seccomp-BPF to protect containers from privilege escalation attacks. Seccomp-BPF restricts system calls made by containerized applications, thereby reducing the attack surface. Sprofiler combines static and dynamic analysis to create tailored seccomp profiles based on application behavior. Evaluation results demonstrate Sprofiler's ability to significantly reduce the number of allowed system calls and mitigate vulnerabilities. However, limitations include potential gaps in dynamic analysis coverage, time-consuming analysis processes, and the need for scalability performance evaluations. Also, specific vulnerabilities addressed and the impact on containerized application functionality are not detailed.

23) PROF-GEN TOOL

A proposed method [57] utilizes the Prof-gen tool to generate a access list of system calls. This method combines dynamic analysis with static binary analysis without needing prior knowledge of container behavior. The user first provides a container image and runs the Prof-gen command for the image. Prof-gen traces the container boot-up process through dynamic analysis and extracts binary files for further analysis. Prof-gen then performs static binary analysis on these extracted binaries and shared libraries inside the image. Through this analysis, Prof-gen restores local symbols and performs call graph analysis without the help of any source code or external tools. During this graph analysis, Prof-gen identifies all the system calls the container requires. A access list of these system calls is generated and used as an option for the docker run command. The tool focuses

on reducing the risk of attacks that exploit vulnerabilities in container isolation. Unlike existing methods, Prof-gen doesn't require prior knowledge of a container's behavior, such as test suites or source codes. Through this mechanism, Prof-gen filters the system, calls for containers, and reduces the attack surface. Results of the evaluation show a reduction in the risk of attacks by 20.2% without causing any issues in the tested containers.The Prof-gen is subject to several limitations that impact its effectiveness and reliability. One notable concern is the false positive and negative rate, which stands at 6.9% for detection mechanisms, indicating a level of inaccuracy that could undermine the tool's utility. Moreover, false negatives, particularly noticeable in Java runtime-based containers, highlight potential limited vision spots in Prof-gen's detection capabilities, potentially leaving vulnerabilities unaddressed. Furthermore, Prof-gen does not possess the ability to address new types of attacks dynamically, leaving systems vulnerable to emerging threats. Additionally, the Seccomp feature within Prof-gen's arsenal is constrained by a blocklist blocking only 44 system calls, potentially limiting its ability to mitigate security risks comprehensively. Lastly, existing studies reveal a need for additional knowledge specific to containers, suggesting gaps in Prof-gen's understanding of containerized environments and associated security challenges.

24) saBPF

Secure Audit BPF (saBPF) [16], an extension of the eBPF framework, enhances system-level auditing in container-based cloud computing. It includes an intrusion detection system and lightweight access control mechanisms, ensuring process and kernel isolation within containers. The methodology involves extending eBPF map helpers to enable the manipulation of kernel object local storage securely. Performance analysis compares saBPF with reference-monitor-based auditing, considering the overhead of running LSM and eBPF together. However, deploying saBPF may be challenging due to the extensive kernel modifications required. Policy conflict resolution across the cgroup hierarchy isn't addressed, potentially limiting access control effectiveness. While saBPF offers secure auditing and simple access control, it may not suffice for complex access control needs or policy conflicts in container environments. Despite aiming for comparable performance to other audit systems, there may still be some performance impact. Overall, saBPF improves container auditing and access control but has limitations regarding kernel modifications, policy resolution, and potential performance impact.

25) Lic-Sec

The proposed methodology [58] involves combining two Linux Security Modules (LSMs), Docker-sec and a modified version of LiCShield, to create a new AppArmor profile generator known as Lic-Sec. This new tool aims to enhance Docker container security by implementing stronger mandatory access control and automating the profile configuration process. Lic-Sec works by generating initial AppArmor profiles and then testing them against a set of known exploits to gauge their effectiveness against zero-day vulnerabilities. Evaluation results indicate that Lic-Sec provides better protection against privilege escalation attacks compared to Docker-sec alone. However, there are limitations to consider: the evaluation is based on a limited set of exploits, potentially not fully representing real-world attack scenarios; its effectiveness against other types of attacks is not explicitly discussed; performance in large-scale deployments is not addressed; and its's no comparison with other security solutions for context.

26) TZ-CONTAINER

The TrustZone-container (TZ-Container) [59] is a secure container approach built on TrustZone technology, designed to shield containers from untrusted operating systems. It creates isolated execution environments (IEEs) for each container process, ensuring their own protected memory space separate from the underlying OS and other processes. Security checks are enforced on system calls within these environments, using semantic analysis and mode switching between user and kernel modes. The TZ-Container utilizes TrustZone technology to establish isolated execution environments (IEEs) for every container process. This ensures that each IEE operates in its own protected bubble, keeping its memory and CPU context isolated from both other processes and the underlying operating system. Implemented on the Hikey development board, TZ-Container can run unmodified Docker images from existing repositories. Evaluation results indicate a performance overhead of around 5%, which is considered acceptable given the security benefits. Importantly, it doesn't require hardware modifications or changes to existing container images. However, the paper has some limitations. It focuses solely on protecting containers from external threats, overlooking potential risks within the container itself. Additionally, evaluation is limited to the Hikey board, leaving uncertainty about its performance on other hardware platforms. There's also a lack of detailed analysis regarding remaining security vulnerabilities within the TZ-Container implementation. Furthermore, while the performance overhead is reasonable for many scenarios, it might be significant in environments where performance is critical. Lastly, the scalability of TZ-Container isn't thoroughly discussed, especially in scenarios with multiple containers running on a single host.

27) BPFCONTAIN

Introduce BPFCONTAIN [60], a container confinement mechanism designed to enhance security and integrate seamlessly with existing container management systems. BPFCONTAIN addresses weak container isolation, providing robust confinement against attacks like namespace escapes and privilege escalation. Its design allows for the kernel to be dynamically hardened. BPFCONTAIN works by leveraging eBPF and a YAML-based policy language to

define container rules. It ensures safety by running eBPF code through a bytecode verifier and maintaining process and container maps to track their state. BPFCONTAIN's deployment doesn't require kernel changes and integrates well with Open Container Initiative-compliant systems, offering strong yet flexible confinement. However, the paper acknowledges some limitations. BPFCONTAIN faces challenges in policy complexity due to eBPF constraints and relies on existing container management systems. There's also a need for further evaluation in real-world scenarios and exploration of scalability and performance in large-scale deployments. Further research need to overcoming policy complexity limitations, evaluating integration with various container runtimes, and conducting thorough security assessments to identify any potential vulnerabilities.

28) bpfbox

bpfbox [61] is a process confinement tool built with eBPF. It aims to offer straightforward, efficient, and adaptable confinement for cloud workloads and beyond. It provides confinement at different levels, like userspace and system call boundaries, offering a granularity not seen in existing mechanisms. The confinement policies are crafted in a user-friendly language suited for ad hoc purposes. With less than 2000 lines of kernel space code, bpfbox remains lightweight. Despite its benchmarks showcasing performance and effectiveness, bpfbox is still a research prototype. This means it may have yet to discover limitations and room for improvement. While specific limitations aren't outlined, potential scalability issues, compatibility challenges with various applications, and performance overheads are worth exploring. Additionally, the simplicity of its policy language might limit its adaptability to complex scenarios, posing challenges for specific use cases. These limitations could vary based on user needs, necessitating further testing and refinement for practical deployments.

29) NNASC

The methodology Neural Networks Analyzing System Calls (NNASC) [62] revolves around employing neural networks to sift through system call data within container setups, aiming at spotting anomalies. Sysdig is utilized to record system calls, benefiting from its container-specific support and kernel module on the host machine. At its core, the method employs a neural network trained to anticipate the subsequent file system path based on the last one used by a system call. To prepare the data for anomaly detection, any words outside the dictionary are replaced with ''unk''. The focus is on single-node workloads within shared enterprise environments rather than distributed setups. The paper delves into the benefits and drawbacks of detecting anomalies in file and directory paths linked to system calls, elaborating on the network architecture and optimization techniques. However, some limitations are acknowledged. The concentration on single-node setups might limit the applicability of the findings to other container environments. Additionally, the evaluation primarily homes in on anomalies in file and directory paths, potentially overlooking other anomaly types. Furthermore, there's a lack of in-depth analysis regarding the method's performance and accuracy in detecting anomalies. Moreover, the chosen threshold value for classification via RMSE (Root Mean Square Error) may not universally suit all applications, possibly leading to misclassification. Lastly, the paper doesn't delve into the computational overhead and resource demands of deploying the method in real-world container environments.

30) OTHERS CONTAINERIZATION SECURITY SOLUTIONS

This section explores additional security solutions in containerization that address the challenges of the shared kernel architecture. However, these solutions fall outside the parameters discussed earlier.

Reference [63] reviewed defense mechanisms for cloud-native systems using static analysis, identifying gaps in existing solutions but requiring further evaluation and validation of proposed mechanisms. Reference [64] proposed the DCIDS framework to enhance anomaly detection in container platforms using machine learning techniques. While their framework achieved superior detection results, the lack of automated actions in alert occurrence and the requirement for extra tools to track application actions causing alerts suggest avenues for further research. Reference [65] proposed KUNERVA for automated network policy discovery in container environments, effectively generating security policies but lacking focus on policy conflict resolution mechanisms. Reference [66] developed Virtuoso for enhancing VM networking, achieving high resource utilization and performance isolation but facing challenges in providing high resource utilization for microsecond-scale workloads. Reference [67] developed μDetector for intrusion detection in microservice apps, addressing the lack of intrusion detection tools targeting microservices but leaving room for further research in real-world deployment challenges. Reference [9] discussed containerization technologies for scientific applications and proposed taxonomies, addressing challenges in distributed computing infrastructures. However, their discussion lacked depth on the limitations of Docker in complex infrastructures and the impact of restricted API access, authentication, and RBAC-based authorization, indicating a research gap in understanding these aspects. Reference [68] proposed state machine models for anomaly detection in Kubernetes clusters, achieving high accuracy in detecting attacks. However, the lack of explanation on interpreting decisions made by neural networks highlights a limitation that warrants further investigation. Reference [69] explored DevOps pipeline attacks and proposed mitigation techniques. Still, the lack of detailed discussion on specific attack mitigation strategies and the effectiveness of proposed protection mechanisms suggests the need for further research. Reference [70] investigated container-based honeypots and proposed a solution but lacked a field study on

TABLE 2. Common attack scenarios in shared kernel environments and suggested solutions.

Attack Scenario	Description	Impacted Layer	Suggested Solution
Container Breakout	Attackers gain unauthorized access from within a container to the host OS or other containers.	Kernel Layer	Implement seccomp, cgroups, and namespaces for restricting container capabilities; Employ AppArmor or SELinux for enhanced access control and policy enforcement.
Privilege Escalation	Exploiting vulnerabilities in the container runtime or kernel to gain elevated access.	Container Layer	Regularly update container runtimes; Use minimal base images; Apply security patches; Enforce the principle of least privilege using LSMs like AppArmor.
Side-Channel Attacks	Extracting sensitive information by exploiting shared resources such as CPU cache.	Kernel Layer	Deploy cache partitioning techniques; Use hardware isolation features like Intel SGX; Enable kernel-based isolation mechanisms such as KPTI to mitigate speculative execution attacks.
Insecure Networking	Unauthorized interception or manipulation of container network traffic.	Networking Layer	Implement network policies to control traffic flow; Use encrypted communication channels (TLS); Isolate container networks with firewall rules and virtual networks.
Image Tampering	Insertion of malicious code or configurations into container images.	Container Image & Registry	Utilize image signing and verification tools (e.g., Notary); Regularly scan images for vulnerabilities before deployment; Use trusted registries and enforce image provenance.

their effectiveness, indicating a need for further research. Reference [71] surveyed container runtime vulnerabilities and proposed a user namespace defense to prevent runtime escapes. Despite demonstrating the effectiveness of their defense mechanism in stopping container escape exploits, they acknowledged limitations such as containers still running as the default root user, impacting defense effectiveness. Moreover, they did not extensively explore the impact of container runtime vulnerabilities, leaving a research gap regarding the comprehensive understanding of such vulnerabilities. Reference [72] proposed the DECH attack pattern and countermeasures for cloud platform security, focusing on combining SGX and TDE to counter the DECH attack. While their approach showed promise in enhancing cloud security, the lack of discussion on scalability and performance impact and limited exploration of potential vulnerabilities and attack scenarios suggests further research in this area. Reference [73] analyzed co-residency attacks on containers and proposed detection methods. Although they demonstrated high success rates for co-residency detection, the sensitivity to various factors and lack of discussion on a specific research gap indicates the need for further investigation. Reference [74] proposed an unsupervised detection and response mechanism for application-layer attacks, effectively reducing attack impact but lacking discussion on scalability and adaptability. Reference [12] surveyed enhancing container security through kernel mechanisms, identifying challenges in adapting LSM modules to containers and highlighting the need for further research. Reference [75] proposed a defense mechanism against privilege escalation attacks in containers, but its limitation in identifying potential exploits suggests the need for further investigation.

Table 2 categorizes potential attack scenarios, ranging from container breakouts and privilege escalations to side-channel attacks and insecure networking. For each scenario, it identifies the impacted layer within the container stack and proposes specific, actionable solutions to mitigate these threats. By addressing these issues systematically, Table 2 not only underscores the importance of robust security practices in containerized deployments but also serves as a reference for researchers and practitioners seeking to enhance the security posture of their containerized applications in shared kernel environments.

B. COMPARATIVE ANALYSIS OF STATE-OF-THE-ART CONTAINERIZATION SECURITY SOLUTIONS

This subsection critically analyzes containerization security solutions, building on the thematic taxonomy outlined in Section III. Table 3 presents an overview of this comparison and offers a structured view of the current state-of-the-art security solutions in container environments.

1) TYPES OF CONTAINER

The distinction between system containers and application containers is crucial when it comes to implementing security measures. System containers encapsulate entire operating system environments, allowing multiple system-level processes to run simultaneously. They often require broader access to system resources and higher resource allocation. On the other hand, application containers focus on running individual applications or microservices within isolated environments. They prioritize isolating application components and typically require fewer resources than system containers. Although both types of containers use techniques like namespaces and control groups (cgroups) for resource isolation, system containers may have weaker isolation than application containers due to their broader scope and access to system resources.

TABLE 3. Comparison of state-of-the-art container security solutions in the shared kernel architecture.

Solutions	Type of Cont.		Security Purpose					Attack Surface				Threat Actors				Threats							Security Iso.			Sec. Sol.	
	SC	AC	I	P	RE	A	Mt	CIRL	CL	OL	KL	IA	IC	EC	MH	PE	CE	IT	IC	IN	MSA	DB	PI	MI	NI	HB	SB
[35]	×	✓	✓	×	✓	✓	✓	✓	✓	×	×	✓	✓	✓	✓	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[36]	✓	×	×	×	×	×	✓	×	✓	×	✓	✓	✓	×	×	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[37]	×	✓	✓	×	✓	×	×	✓	✓	×	✓	✓	✓	×	×	✓	✓	×	×	×	✓	✓	×	✓	×	×	✓
[38]	×	✓	✓	×	✓	×	×	×	×	×	✓	✓	✓	×	×	✓	✓	×	×	×	✓	×	×	✓	×	×	✓
[39]	✓	✓	✓	✓	×	×	×	×	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	✓	✓	×	✓
[40]	✓	×	✓	✓	×	×	×	✓	✓	×	×	✓	✓	✓	✓	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[41]	✓	×	✓	✓	✓	✓	×	✓	✓	✓	×	✓	✓	✓	✓	✓	✓	×	×	×	×	×	×	✓	×	×	✓
[42]	✓	×	✓	×	×	×	×	×	✓	✓	✓	✓	✓	✓	✓	×	✓	✓	✓	×	×	×	✓	×	×	×	✓
[43]	✓	×	✓	×	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	×	×	✓	×	×	✓
[44]	×	✓	✓	✓	✓	×	×	✓	✓	×	×	✓	✓	×	×	×	✓	×	×	×	✓	✓	✓	×	×	×	✓
[45]	✓	✓	✓	×	✓	×	✓	×	✓	✓	×	✓	✓	✓	×	✓	✓	✓	✓	✓	×	×	×	×	✓	×	✓
[46]	✓	✓	×	×	✓	✓	×	✓	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	×	×	×	✓	×	✓	×
[47]	✓	✓	✓	×	✓	×	×	✓	✓	×	×	✓	✓	✓	×	✓	✓	✓	✓	×	×	×	✓	×	×	×	✓
[48]	✓	✓	✓	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	×	×	✓	✓	✓	×	×	×	✓	×	×	×	✓
[49]	✓	×	✓	×	×	×	×	×	✓	×	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[50]	✓	×	✓	×	✓	×	×	✓	✓	×	×	✓	✓	×	✓	×	✓	×	×	×	✓	×	✓	×	×	✓	✓
[51]	✓	✓	✓	×	×	×	×	×	×	×	✓	×	✓	✓	×	✓	✓	✓	✓	×	✓	×	×	✓	×	✓	×
[52]	✓	✓	✓	✓	✓	×	×	×	✓	✓	✓	✓	✓	×	✓	✓	✓	✓	✓	×	✓	✓	×	✓	×	✓	×
[53]	×	✓	✓	×	✓	×	×	✓	✓	×	×	✓	✓	×	×	✓	✓	×	×	×	×	×	✓	×	×	✓	×
[54]	✓	×	✓	✓	✓	✓	×	×	×	×	✓	✓	✓	×	✓	✓	✓	×	×	×	✓	×	×	✓	×	×	✓
[55]	✓	✓	×	×	✓	×	×	✓	✓	×	✓	Na	Na	Na	Na	✓	✓	✓	✓	×	×	✓	×	×	✓	✓	×
[56]	×	✓	✓	×	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	×	✓	✓	×	×	×	✓
[57]	×	✓	✓	×	×	×	×	✓	✓	×	×	✓	✓	✓	×	✓	✓	×	✓	×	×	×	✓	×	×	×	✓
[16]	✓	×	✓	✓	×	×	×	✓	✓	×	×	✓	✓	×	✓	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[58]	×	✓	Na	Na	Na	Na	Na	×	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	✓	×	✓	×	✓	×	×	×	✓
[59]	×	✓	✓	×	✓	×	×	✓	✓	✓	×	×	✓	×	✓	✓	✓	✓	×	✓	✓	✓	×	✓	×	✓	✓
[60]	✓	✓	✓	×	✓	×	×	✓	✓	×	×	×	✓	✓	×	✓	✓	✓	✓	×	×	×	×	✓	×	×	✓
[61]	✓	×	✓	×	✓	×	×	✓	✓	×	✓	✓	✓	✓	✓	✓	✓	×	×	×	×	×	✓	×	×	×	✓
[62]	✓	×	✓	×	×	×	×	✓	✓	×	×	✓	✓	×	×	×	✓	×	×	×	×	✓	✓	×	×	×	✓

Abbreviations and Symbols:

SC: System Container AC: Application Container I: Isolation P: Portability RE: Resource Efficiency A: Agility Mt: Multi-tenancy CIRL: Container Image & Registry Layer CL: Container Layer OL: Orchestration Layer KL: Kernel Layer IA: Internal Application IC: Inter Container EC: External Container MH: Malicious Host OS PE: Privilege Escalation CE: Container Escape IT: Image Tampering IC: Insecure Configuration IN: Insecure Networking MSA: Meltdown & Spectre Attacks DB: Data Breach PI: Process Isolation MI: Memory Isolation NI: Network Isolation HB: Hardware Based SB: Software Based Na: Not available

NIMOS [36], KRSIE [40], SandBPF [41], SySched [42], Seccomp-eBPF [43], PBRIEM [49], ContainerGuard [50], GyroidOS [54], saBPF [16], bpfbox [61], and NNASC [62] are proposed security mechanisms for system containers, while sysE [35], μSWITCH [37], Confine [38], CODA [44], HSCPA [53], Sprofiler [56], Prof-gen [57], Lic-Sec [58], and TZ-Container [59] are proposed security solutions for application containers. Additionally, SysVerify [39], Bastion+ [45], CIMM [46], PACED [47], FIT framework [48], PPASes [51], TCX [52], CCSPS [55], and BPFContain [60] are proposed mechanisms that aim to provide security enhancements for both types of containers, addressing the specific requirements and characteristics of each container type. These efforts ensure the integrity and reliability of containerized environments in diverse deployment scenarios.

2) SECURITY PURPOSE IN CONTAINERIZATION

When discussing security purposes in containerized environments, it is essential to grasp each security measure's specific objectives. Isolation ensures that each container operates independently, safeguarding against unauthorized access or tampering between containers. Portability ensures that applications can transition smoothly between different environments without compromising security, facilitating efficient adaptation and scalability for businesses. Resource efficiency maximizes utilizing computing resources within containers to optimize performance and minimize vulnerabilities. Agility enables quick deployment and adaptation of applications without sacrificing security, seamlessly integrating security measures into the development process. Multi-tenancy allows secure infrastructure sharing among multiple users or applications, emphasizing strong isolation and robust access control and resource allocation support.

In container environments, prioritizing isolation, portability, resource efficiency, agility, and multi-tenancy is crucial for comprehensive security while facilitating flexible and efficient operations across different sets. Table 3 compares various security solutions for containerization, highlighting their focus on isolation, portability, resource efficiency, agility, and multi-tenancy.

sysE [35] emphasizes isolation, resource efficiency, agility, and multi-tenancy but lacks focus on portability. NIMOS [36] prioritizes multi-tenancy but doesn't emphasize other security purposes significantly. μSWITCH [37] and Confine [38] focus on isolation and resource efficiency, while SysVerify [39] and KRSIE [40] prioritize isolation and portability. SandBPF [41] addresses all security purposes comprehensively, emphasizing isolation, portability, resource efficiency, and agility but not multi-tenancy. SySched [42] primarily focuses on isolation, while Seccomp-eBPF [43] emphasizes isolation and resource efficiency but lacks emphasis on portability, agility, or multi-tenancy. CODA [44] balances isolation, portability, and resource efficiency but doesn't prioritize agility or multi-tenancy. Bastion+ [45] concentrates on isolation, resource efficiency, and multi-tenancy but lacks focus on portability or agility. CIMM [46] highlights resource efficiency and agility but not isolation, portability, or multi-tenancy. PACED [47] prioritizes isolation and resource efficiency but lacks emphasis on portability, agility, or multi-tenancy. The FIT framework [48] emphasizes isolation and portability but not resource efficiency, agility, or multi-tenancy.

PBRIEM [49], ContainerGuard [50], [50], PPASes [51], HSCPA [53], CCSPS [55], Sprofiler [56], Prof-gen [57], saBPF [16], Lic-Sec [58], TZ-Container [59], BPFContain [60], bpfbox [61], and NNASC [62] have varying degrees of emphasis on isolation, resource efficiency, and other security purposes. These distinctions are important for organizations to choose the most suitable security solution based on their specific requirements and priorities in containerized environments.

3) ATTACK SURFACE

Understanding and mitigating vulnerabilities at each layer of the Attack Surface is crucial. It empowers organizations to reinforce their container security and effectively protect against potential threats. The unique security challenges at each layer, such as outdated software in container images, runtime vulnerabilities, weaknesses in orchestration components, and kernel exploits, can be addressed with tailored security measures like regular patching, secure configurations, and access controls.

Table 3 compares attack surfaces across different layers of containerization for various security solutions. At the Container Image & Registry Layer, sysE [35], NIMOS [36], μSWITCH [37], SandBPF [41], CODA [44], CIMM [46], FIT framework [48], ContainerGuard [50], CCSPS [55], saBPF [16], TZ-Container [59], BPFContain [60], and bpfbox [61] exhibit vulnerabilities, indicating potential attack vectors within the container image and registry layer. Regarding the Container Layer, NIMOS [36], μSWITCH [37], SysVerify [39], KRSIE [40], SandBPF [41], SySched [42], Seccomp-eBPF [43], CODA [44], Bastion+ [45], CIMM [46], PACED [47], FIT framework [48], PBRIEM [49], ContainerGuard [50], TCX [52], HSCPA [53], CCSPS [55], Sprofiler [56], Prof-gen [57], saBPF [16], TZ-Container [59], and bpfbox [61] are susceptible to attacks, highlighting potential weaknesses within the container layer itself. Concerning the Orchestration Layer, NIMOS [36], SysVerify [39], KRSIE [40], SySched [42], Seccomp-eBPF [43], CIMM [46], PACED [47], FIT framework [48], HSCPA [53], CCSPS [55], Sprofiler [56], saBPF [16], Lic-Sec [58], TZ-Container [59], BPFContain [60], bpfbox [61], and NNASC [62] demonstrate vulnerabilities, suggesting potential risks within the orchestration layer. Finally, at the Kernel Layer, sysE [35], NIMOS [36], μSWITCH [37], Confine [38], SysVerify [39], KRSIE [40], SySched [42], Seccomp-eBPF [43], CODA [44], CIMM [46], PACED [47], FIT framework [48], PBRIEM [49], CCSPS [55], Sprofiler [56], saBPF [16], TZ-Container [59], and bpfbox [61] show vulnerabilities, indicating possible security threats at the kernel layer.

4) THREAT ACTORS

The Threat Actors in Table 3 present the different entities that pose security risks within containerized environments. These actors encompass various scenarios, including internal application threats involving vulnerabilities or malicious actions within the containerized application, while inter-container risks arise from interactions between separate containers, which could lead to unauthorized access or data breaches. External container threats originate from containers external to the environment and may exploit vulnerabilities or launch attacks against the containerized system. Additionally, concerns regarding a malicious host OS involve security risks stemming from compromises or malicious activities within the underlying host OS, potentially resulting in unauthorized access or manipulation of container resources. Recognizing these distinct threat actors allows organizations to develop targeted security strategies to mitigate risks and uphold the security of containerized environments. The existing solutions NIMOS [36], μSWITCH [37], Confine [38], CODA [44], HSCPA [53], and NNASC [62] provide security measures against threats from internal applications and container interactions but lack specific measures for external container threats or malicious host OS. sysE [35], SysVerify [39], KRSIE [40], SandBPF [41], SySched [42], Seccomp-eBPF [43], CIMM [46], PBRIEM [49], Sprofiler [56], Lic-Sec [58], and bpfbox [61] offer comprehensive security measures covering threats from all four threat actors.

Other methods like ContainerGuard [50], GyroidOS [54], TCX [52], and saBPF [16] focus on addressing internal application threats, interactions between containers, and potentially compromised host operating systems but may not prioritize security against external container threats. Bastion+ [45], PACED [47], FIT framework [48], and Profgen [57] offer robust protection against internal applications and interactions between containers and external containers but do not specifically address threats from malicious host operating systems. PPASes [51], and BPFContain [60] prioritizes security against interactions between containers and external containers but may lack measures against internal application threats or malicious host OS. Methods like TZ-Container [59] offer security against threats from container interactions and potentially compromised host operating systems but do not address threats from internal applications or external container threats.

However, some methods like CCSPS [55] need more information to determine their specific security focuses. Each technique presents unique security measures tailored to mitigate particular threats, allowing organizations to select the most appropriate approach based on their security requirements and priorities within containerized environments.

5) THREATS

The next parameter in the scenario of security challenges of containerization within a shared kernel environment is threats. Threats are potential risks or dangers that can exploit weaknesses in a system, posing harm or causing disruptions. These risks can originate from malicious individuals, software flaws, hardware malfunctions, or environmental factors. Threats originating in containerization due to shared kernel architecture may include privilege escalation, where unauthorized access levels are gained, and container escape, allowing attackers to breach container boundaries and access the underlying host system. Image tampering involves unauthorized modifications to container images, potentially compromising the integrity of applications. Insecure configurations and networking expose vulnerabilities, while Meltdown and Spectre attacks exploit hardware vulnerabilities, risking data leakage. Ultimately, data breaches occur when sensitive information within containers is compromised. Mitigating these threats necessitates diligent security practices, including regular assessments, secure configurations, and robust network security measures. The security solutions are each designed to address specific threats within containerized environments. For instance, sysE [35] tackles privilege escalation, and container escape risks but doesn't address image tampering or insecure configurations. On the other hand, methods like PPASes [51] and TCX [52] aim to mitigate a broader range of threats, including privilege escalation, container escape, image tampering, and insecure networking. Some methods, such as Sprofiler [56], primarily focus on addressing data breach risks, while others, like Bastion+ [45] and CIMM [46] offer comprehensive solutions covering multiple threats including privilege escalation, container escape, image tampering, and insecure configurations. However, not all methods address every potential threat, as seen with bpfbox [61], focusing mainly on privilege escalation and container escape without addressing image tampering or insecure configurations. While the methods discussed offer varying security measures, it's crucial to note that none alone can ensure complete protection in shared kernel architecture. The limitations of individual approaches underscore the need for a more comprehensive defense mechanism. Such a mechanism should address all security aspects in a generalized manner, providing a robust and adaptable security framework to safeguard container environments within shared kernel architecture effectively.

6) SECURITY ISOLATION

Security isolation is a fundamental aspect of containerization security. It refers to securely segregating and protecting containerized processes and resources. This isolation is crucial as it ensures that each container operates independently within its secure environment. It minimizes the risk of interference or compromise between containers, thereby enhancing the overall security of the containerized environment.

In a shared kernel architecture, where containers share the same kernel of the host operating system, different types of isolation play crucial roles in ensuring a secure containerized environment. Process isolation involves running container processes independently, preventing one container from interfering with another. This isolation is typically

achieved through Linux namespaces, which create separate process namespaces for each container. Memory isolation securely partitions memory resources, ensuring containers cannot access each other's memory. Techniques such as control groups (cgroups) limit the amount of memory each container can use, and technologies like Memory Resource Management (MRM) provide further control over memory allocation and usage. Similarly, network isolation creates separate network environments for each container, preventing unauthorized access to network resources. Containerization platforms like Docker and Kubernetes can provide a secure, shared kernel architecture environment by employing these isolation mechanisms. These isolation mechanisms contribute to a secure containerized environment, protecting against various security threats and vulnerabilities.

Table 3 assesses existing solutions for implementing security isolation within containerization environments. These methods are evaluated based on their processing, memory, and network isolation approach. Each method prioritizes different aspects of security isolation, with some focusing more on process isolation, others on memory isolation, and a few on network isolation. For instance, methods like sysE [35], NIMOS [36], KRSIE [40], SySched [42], CODA [44], PACED [47], FIT framework [48], PBRIEM [49], ContainerGuard [50], HSCPA [53], Sprofiler [56], Prof-gen [57], saBPF [16], Lic-Sec [58], bpfbox [61], and NNASC [62] prioritize process isolation, while μSWITCH [37], Confine [38], SandBPF [41], Seccomp-eBPF [43], CIMM [46], PPASes [51], TCX [52], GyroidOS [54], TZ-Container [59], and BPFContain [60] emphasize memory isolation. Conversely, methods like Bastion+ [45] and CCSPS [55] prioritize network isolation, and SysVerify [39] implement memory and network isolation but lacks process isolation. This reveals a diverse landscape of methods for implementing security isolation in containerization environments, each with strengths and limitations. While many methods prioritize process isolation, it's important to note that more is needed for comprehensive security. Memory and network isolation are equally critical, yet not all methods prioritize them adequately.

7) TYPES OF CONTAINER SECURITY SOLUTION

In a shared kernel architecture, where multiple containers share the same kernel of the host operating system, ensuring robust isolation is paramount for security. This isolation can be achieved through two main mechanisms: hardware-based and software-based solutions. Hardware-based solutions typically rely on features provided by the underlying hardware, such as virtualization technologies like Intel VT-x or AMD-V, which enable the creation of isolated execution environments known as virtual machines (VMs). These VMs have independent kernels and memory spaces, providing strong isolation between containers. On the other hand, software-based solutions leverage operating system features and containerization technologies to achieve isolation. Linux namespaces and control groups (cgroups) are fundamental components in software-based isolation. Linux namespaces isolate various aspects of the operating system, such as processes, network resources, and file systems, creating a virtualized environment for each container. Control groups, on the other hand, manage and limit the resource usage of containers, including CPU, memory, and disk I/O, ensuring fair allocation and preventing one container from monopolizing shared resources.

Among the proposed solutions mentioned, CIMM [46], ContainerGuard [50], PPASes [51], TCX [52], HSCPA [53], CCSPS [55], and TZ-Container [59] focus on hardware-based security solutions. These solutions utilize hardware features such as virtualization extensions or dedicated security processors to enhance container isolation and security. On the other hand, other solutions provide software-based security mechanisms, leveraging Linux namespaces, cgroups, and other operating system features to achieve container isolation within a shared kernel architecture. Both hardware-based and software-based solutions have their advantages and trade-offs. Hardware-based solutions often offer stronger isolation and security guarantees due to the dedicated hardware support, but they may come with higher overhead and complexity. Software-based solutions, while more lightweight and flexible, may be limited by the capabilities of the underlying operating system and hardware. Ultimately, the choice between hardware-based and software-based solutions depends on factors such as performance requirements, deployment environment, and the specific security needs of the containerized applications.

V. QUANTITATIVE COMPARISON OF STATE-OF-THE-ART CONTAINERIZATION SECURITY SOLUTIONS

In this section, we provide a structured and detailed analysis of the latest containerization security solutions, based on quantitative data. This analysis is divided into distinct subsections to offer a clearer understanding of the effectiveness and efficiency of these solutions in addressing security concerns within containerized environments.

Containerization introduces specific challenges, particularly in relation to maintaining security within a shared kernel environment. Recent advancements in container security solutions have introduced a variety of strategies aimed at mitigating these challenges. To comprehensively evaluate these solutions, we analyze the following key attributes in our comparison: Performance Overhead, Attack Type Coverage, False Positive/Negative Rates, Attack Surface Reduction, Dynamic Adaptability, Resource Utilization, Integration with Existing Protocols, and the Underlying Technologies utilized.

Each of these attributes is discussed in dedicated sections to facilitate a deeper understanding of the trade-offs, benefits, and potential limitations associated with the various containerization security solutions.Table 4 provides a detailed quantitative comparison of these solutions, focusing on critical aspects necessary for assessing their effectiveness, resource usage, and adaptability in dynamic environments.

TABLE 4. Quantitative comparison of state-of-the-art containerization security solutions and their performance overheads.

Method	Performance Overhead	Attack Type	FP/NR	ASR	DA	RU	Integration	Technologies
sysE [35]	1.28%	Multiple	0.645-0.773	↑7.3-21.5%	✓	Optimized	TinyML-based	eBPF
NIMOS [36]	3.20%	Container escape	High FP	NM	✓	NM	NM	Seccomp
μSWITCH [37]	↓32.7-98.4%	Unauthorized	NM	↓32.7-98.4%	NAD	↑4.9x	Intel® MPK	Intel MPK, Wasm, PI
Confine [38]	0.50%	Privilege	NM	NM	NAD	NM	NM	Static analysis
SysVerify [39]	<1%	Privilege	NM	Significant ↓	✓	Low CPU	NM	Dynamic & Static
KRSIE [40]	1.50%	Mining pool	NM	NM	✓	NM	SELinux, AppArmor	eBPF
SandBPF [41]	0-10%	CVE exploits	0-10%	CVE Prevent	✓	NM	eBPF coexist	eBPF
SySched [42]	NM	Co-location	NM	↑20%	NAD	NM	SELinux, AppArmor	Seccomp
Seccomp-eBPF [43]	↑55.40%	Real-world	↓55.4%	↓55.4%	✓	↑10%	Kernel privileges	Advanced eBPF
CODA [44]	0.7-5%	CPU-exhaustion	↓ with threshold	NM	✓	Low	NM	eBPF
Bastion+ [45]	↑20.6%	Network Topology	NM	Isolation	✓	2.3% CPU	AppArmor, SELinux	eBPF
CIMM [46]	NP	Multiple	0.1-0.2%	NM	NAD	Low	NM	eBPF, HW features
PACED [47]	<1%	Container-escape	No FN	NM	✓	Minimal	NM	Provenance
FIT framework [48]	5-10%	Co-residency	NM	NM	✓	NM	SELinux, AppArmor	Spec-based
PBRIEM [49]	3.50%	DoS	NP	Quantified	NP	cAdvisor	Linux NS	Process NS
ContainerGuard [50]	4.5%	Side-channel	Varies	NM	✓	4.50%	Seamless	VAEs
PPASes [51]	<15%	Memory-based Iago	NM	Direct prevent	NAD	Modest	NM	Arm HW, SGX, TrustZone
TCX [52]	5.77%	Adversary 1&2	NM	Enclave	NAD	1GB SC-VM	NM	AMD SEV
HSCPA [53]	3.20%	Mimicry	NP	Empirical ↓	✓	NM	SELinux, AppArmor	Dynamic & Static
GyroidOS [54]	Variable	Kernel surface	NM	8.6% ↓	✓	Negligible	NM	Seccomp, BPF
CCSPS [55]	NP	Multiple	0.1-0.2%	NM	✓	Low	NM	eBPF
Sprofiler [56]	<1%	Privilege	NP	Significant ↓	✓	NM	Seccomp-BPF	eBPF
Prof-gen [57]	↓20.2%	Container	6.9	↓20.2%	NAD	↓20.2%	NM	Syscall trace, Static
saBPF [16]	NP	Intrusion	NM	NM	✓	ProvBPF outperform	Custom	eBPF audit
Lic-Sec [58]	Negligible	9 types	NM	Restrictions	NAD	Negligible	AppArmor	MAC, AppArmor
TZ-Container [59]	5%	Direct	NM	Checks	NAD	5%	NM	ARM TrustZone
BPFContain [60]	<16%	Inter-container	NM	Restrict syscalls	✓	Comparable	LSMs coexist	eBPF
bpfbox [61]	Acceptable	Various	Valid flagged	Restrictions	✓	Slight ↓	Permissive mode	eBPF flexible
NNASC [62]	Reduced	Mimicry	NM	NM	✓	NM	Anomaly NN	NN sys call analysis

Abbreviations and Symbols:

FP/NR: False Positive/Negative Rates ASR: Attack Surface Reduction DA: Dynamic Adaptability RU: Resource Utilization ✓: Addressed or Provided ↑: Increase or Improvement ↓: Decrease or Reduction NM: Not Mentioned NP: Not Provided NAD: Does Not Address Dynamic Adaptation FN: False Negatives NS: Namespace Wasm: WebAssembly PI: Process Isolation HW: Hardware VAEs: Variational Autoencoders SEV: Secure Encrypted Virtualization MAC: Mandatory Access Control

A. PERFORMANCE OVERHEAD

Performance overhead is the additional computational cost incurred when implementing a security solution. Lower overhead percentages are generally more desirable, as they signify less impact on system performance. Maintaining low overhead is essential in real-world applications, where system efficiency and usability are critical for ensuring smooth operations, especially in high-performance environments. When comparing the performance overhead of various proposed container security solutions, there is significant variance in how each solution affects system resources. This variation highlights the need for careful selection of security solutions based on the performance tolerance of the target environment. Figure 4 is a breakdown of how different methods perform regarding overhead.

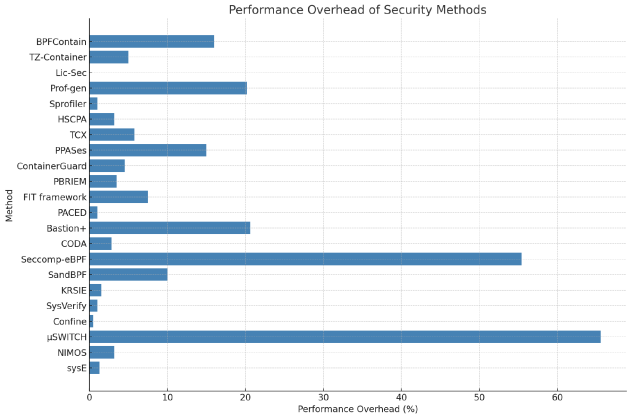

FIGURE 4. Performance overhead of container security solutions.

1) LOW OVERHEAD SOLUTIONS

Several methods stand out for their exceptionally low overhead, making them ideal for performance-sensitive environments. Confine [38] incurs just 0.50% overhead, offering an extremely lightweight solution.SysVerify [39] maintains an overhead of less than 1%, ensuring minimal impact on system performance. KRSIE [40] operates at 1.50% overhead, balancing performance and security. sysE [35], at 1.28%, also demonstrates low overhead while maintaining robust security. CODA [44] shows a flexible overhead ranging from 0.7% to 5%, making it adaptable to different deployment scenarios. PACED [47] and Sprofiler [56] both maintain an overhead of less than 1%, making them ideal for high-performance environments. Lic-Sec [58] boasts negligible overhead, further emphasizing its suitability for performance-critical applications. These solutions are notably efficient and advantageous for settings where system performance is a priority, as they offer strong security while minimizing resource consumption.

2) MODERATE OVERHEAD SOLUTIONS

Some solutions strike a balance between performance and security, offering moderate overhead while still maintaining robust protection.NIMOS [36] operates at 3.20%, positioning itself as a middle-ground option for environments that can tolerate some performance trade-offs. ContainerGuard [50], with approximately 4.5% overhead, similarly balances security with manageable resource consumption. These methods are appropriate for scenarios where moderate performance impact is acceptable in exchange for enhanced security.

3) HIGH OVERHEAD SOLUTIONS

Several solutions impose higher overhead, which may limit their application in performance-sensitive environments, but are still appropriate where security is paramount. The FIT framework [48] incurs 5-10% overhead, making it more suitable for scenarios where security is the primary concern despite performance impact.TCX [52] introduces 5.77% overhead, which could be restrictive in resource-constrained environments but acceptable in security-centric settings. BPFContain [60] operates at less than 16% overhead, and PPASes [51] with less than 15%, both of which provide substantial security enhancements at the cost of higher resource consumption. These solutions are designed for applications where performance is secondary to security concerns.

4) VARIABLE OVERHEAD SOLUTIONS

Several solutions exhibit adaptable performance overhead depending on the deployment context, making them suitable for environments with varying resource requirements. μSWITCH [37] stands out by significantly increasing overhead, ranging from 32.7% to 98.4%, offering enhanced security at the cost of performance in specific scenarios. Similarly, Seccomp-eBPF [43] raises overhead by 55.40%, making it more appropriate for environments where security is prioritized over performance efficiency. Bastion+ [45] shows a 20.6% increase in overhead, indicating a security-focused solution with higher resource consumption. On the other hand, Prof-gen [57] demonstrates a notable 20.2% reduction in overhead, proving that security can be enhanced without necessarily compromising performance. Additionally, GyroidOS [54] demonstrates variable overhead, suggesting its impact can be adjusted based on specific system requirements. bpfbox [61] is described as having ''acceptable'' overhead, indicating that its performance impact remains within manageable levels. Lastly, NNASC [62] showcases reduced overhead compared to traditional virtual machines (VMs), reflecting its efficiency in modern containerized environments.

The variations in performance overhead across these security solutions provide decision-makers with a broad range of options. Whether seeking to minimize resource consumption or optimize security in high-risk environments, decision-makers can select the solution that best aligns with their security and performance trade-off requirements.

B. ATTACK TYPE MITIGATION

The Attack Type describes the specific threats each security method is designed to mitigate. These threats include a broad range of attacks, such as container escapes, privilege escalation, side-channel attacks, and more. Understanding the focus of each solution helps decision-makers select the proper security measures based on the specific vulnerabilities in their containerized environments. Figure 5 provides an overview of the types of attacks each method addresses.

1) MULTI-VECTOR DEFENSE SOLUTIONS

Some security methods are designed to protect against a broad spectrum of attack vectors, offering robust, comprehensive defenses for various deployment environments. sysE [35] addresses multiple attack vectors, including container escapes, privilege escalation, and side-channel attacks, making it a versatile solution for diverse environments. CIMM [46] and CCSPS [55] are similarly architected to defend against a wide range of threats, ensuring robust protection across multiple attack types. Lic-Sec [58] is particularly noteworthy, offering protection against nine distinct attack types, positioning it as one of the most comprehensive solutions. These multi-vector solutions are ideal for environments facing threats requiring strong, all-encompassing security.

2) SPECIALIZED SOLUTIONS FOR CONTAINER ESCAPES

Several methods focus on preventing container escape attacks, critical in maintaining container isolation and preventing unauthorized access to the host system. NIMOS [36] and PACED [47] are designed to thwart container escape attempts, making them highly specialized in ensuring container boundaries are not breached. Prof-gen [57]specifically aims to safeguard container integrity, ensuring that containers remain secure and protected from attacks seeking to break isolation. These methods are highly effective for environments where container isolation is a top priority and protection against escape attacks is critical.

3) SOLUTIONS FOCUSED ON PRIVILEGE ESCALATION

Some methods concentrate on defending against privilege escalation, a common security breach where attackers attempt to gain higher-level access than authorized. μSWITCH [37], Confine [38], SysVerify [39], and Sprofiler [56] focus on preventing unauthorized access escalation, ensuring that attackers cannot exploit privileges to gain control over the container or host system. These methods are particularly effective in environments where protecting against privilege escalation is crucial to maintaining system integrity and preventing unauthorized access.

4) CRYPTO-MINING AND SPECIFIC THREAT SOLUTIONS

Specific solutions are tailored to mitigate specialized threats, such as crypto-mining attacks or vulnerabilities specific to certain environments. KRSIE [40] is explicitly designed to protect against crypto-mining threats, particularly prevalent in environments susceptible to unauthorized mining operations. SandBPF [41] and Seccomp-eBPF [43] are equipped to defend against CVE exploits and real-world vulnerabilities, making them particularly useful in environments where exposure to known vulnerabilities is a significant concern. SySched [42] is tailored to mitigate co-location risks in multi-tenant cloud environments, where resources are shared among multiple users, making this method ideal for cloud-based services.

These specialized solutions are well-suited for environments with unique security needs, such as those prone to crypto-mining or vulnerabilities related to shared resources.

5) RESOURCE DEPLETION AND DOS ATTACK PREVENTION

Some methods specifically focus on preventing resource depletion attacks, such as CPU exhaustion and denial of service (DoS) attacks. CODA [44] and PBRIEM [49] are designed to address CPU exhaustion and DoS incidents, making them essential for ensuring uninterrupted service availability and system performance, especially in high-availability environments. These methods are crucial for maintaining service continuity, particularly when resource depletion attacks could lead to costly downtime or disruptions.

6) SOLUTIONS FOR NETWORK AND COMMUNICATION SECURITY

For environments where network security and communication protection are paramount, specific solutions are designed to secure the structural components of systems and ensure safe communications. Bastion+ [45] provides robust protection for system components and network communication security, making it suitable for securing critical infrastructures. TCX [52] and NNASC [62] address adversarial and mimicry attacks, providing security measures against actors attempting to imitate legitimate processes or exploit the system's trust mechanisms. These methods are ideal for environments where protecting system integrity and communication security is a priority.

7) SIDE-CHANNEL AND MEMORY-BASED EXPLOIT PREVENTION

Several methods focus on defending against side-channel attacks and memory-based exploits, which are increasingly common in modern containerized environments. Container Guard [50] and PPASes [51] provide security measures for sensitive information, defending against indirect attacks like side-channel and memory-based exploits. These solutions are crucial for sensitive data environments, where attacks targeting memory or exploiting side channels could lead to significant security breaches.

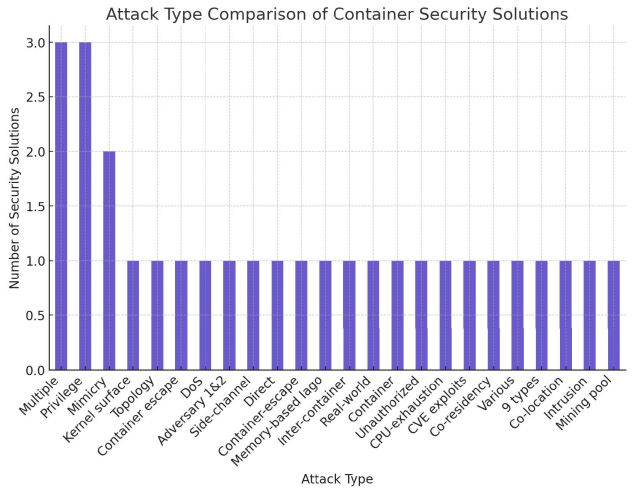

FIGURE 5. Attack type impact comparison of container security solutions.

8) KERNEL VULNERABILITY AND CO-LOCATION RISK MITIGATION

Methods like GyroidOS [54] and TZ-Container [59] offer specialized countermeasures to protect against kernel surface vulnerabilities and direct threats. GyroidOS fortifies systems against kernel surface vulnerabilities, making it particularly useful in environments where kernel attacks are a significant concern. TZ-Container offers specialized countermeasures for direct threats, providing targeted protection for high-risk environments. These solutions are essential for safeguarding core system components and preventing low-level attacks on the operating system kernel.

9) INTER-CONTAINER COMMUNICATION SECURITY

In environments where inter-container communication is a security risk, methods like BPFContain [60] and bpfbox [61] demonstrate strong adaptability: Both solutions provide security measures for handling inter-container communication, making them highly adaptable to security challenges arising from containers interacting within a shared environment. These methods are effective in multi-container environments where securing communication between containers is critical. This array of container security methods spans from expansive multi-vector defenses to highly specialized solutions tailored to specific environments and threat models. Decision-makers must carefully assess their deployment context to choose the most appropriate security solution, balancing broad protection with specialized defenses where necessary.

C. FALSE POSITIVE/NEGATIVE RATES (FP/NR)

The False Positive/Negative Rates (FP/NR) measure the accuracy of a security method in detecting threats. A high false positive rate can result in unnecessary actions and operational disruptions, while a high false negative rate may lead to missed attacks, leaving the system vulnerable. Ideally, container security solutions should maintain a balance, minimizing false positives and negatives to ensure reliability, precision, and efficiency. The following analysis highlights the FP/NR performance of various container security methods, demonstrating their strengths and areas for improvement in detection accuracy.

1) BALANCED FP/NR SOLUTIONS

sysE [35] reports a balanced FP/NR range between 0.645 and 0.773, indicating a reasonable compromise between cautious detection and precision. This range signifies that sysE is designed to avoid excessive false positives while maintaining a high threat detection level, making it a suitable option for general-purpose security needs.Seccomp-eBPF [43] shows a significant improvement in accuracy, with a decreased FP/NR rate of 55.4%, which is substantial for container security. This improvement highlights the method's effectiveness in reducing unnecessary false positives or negatives, enhancing system reliability. These solutions balance overcautious detection and effective threat identification, making them ideal for environments where false alarms and missed threats carry significant operational risks.

2) HIGH FALSE POSITIVE RATE SOLUTIONS

NIMOS [36] is notable for its high false positive rate, which may lead to operational inefficiencies due to the increased number of false alarms. While it offers robust threat detection, the frequent false positives can disrupt workflow and cause unnecessary resource consumption. Prof-gen [57] also reports a relatively high FP rate of 6.9%, which could indicate a cautious approach to security. While it ensures comprehensive threat detection, the high number of false alerts may be a downside in environments where operational efficiency is critical. For environments where false alarms can cause significant disruption or resource strain, these methods may require further tuning or adjustments to improve their detection accuracy.

3) LOW FP/NR RATE SOLUTIONS

CIMM [46] and CCSPS [55] demonstrate meager FP/NR rates, ranging from 0.1% to 0.2%, positioning them as methods with high accuracy. Their ability to minimize false positives and negatives makes them ideal for environments requiring precise threat detection without unnecessary interruptions. SandBPF [41] exhibits a fluctuating FP/NR rate between 0% and 10%, indicating that its reliability may vary depending on the deployment environment and threat landscape. This variability suggests that it can perform well in certain situations but may require further adjustments in others to achieve optimal results. These low FP/NR solutions are highly effective in environments where false positives and negatives must be minimized. They ensure smooth operations without compromising security.

4) SPECIALIZED SOLUTIONS WITH NOTEWORTHY FP/NR RATES

PACED [47] claims no false negatives, ensuring that all potential threats are detected. However, it needs to provide data on its false positive rate, which leaves uncertainty regarding its overall detection accuracy. ContainerGuard [50] demonstrates varying accuracy, with its FP/NR rates potentially contingent upon the specific attack vectors it defends against and how it is configured. This variability makes it adaptable but suggests that its effectiveness may depend on the environment in which it is deployed. These methods excel in specific aspects of detection but may require further data or tuning to ensure consistent accuracy across different threat scenarios.

5) UNSPECIFIED FP/NR RATES

Several methods do not provide specific FP/NR data, leaving their detection accuracy open to interpretation or requiring further investigation:μSWITCH [37], Confine [38], and SysVerify [39] lack specified FP/NR rates, marked as Not Mentioned (NM) or Not Provided (NP). This omission points to a gap in understanding their detection performance, suggesting that further research or disclosure is needed to evaluate their effectiveness fully. While these methods may offer strong security capabilities, the absence of FP/NR data limits a comprehensive assessment of their accuracy in real-world deployment environments.

This analysis highlights the diversity in accuracy among container security methods, with some demonstrating fine-tuned detection capabilities and others leaving room for enhancement or further clarification. When choosing a security solution, the specific FP/NR rates can be just as critical as the type of threat addressed, particularly in environments where the cost of false positives or negatives carries significant implications. Optimally balanced solutions, such as sysE and Seccomp-eBPF, offer a reliable blend of detection accuracy, while methods like CIMM and CCSPS provide high precision with minimal false detections.

D. ATTACK SURFACE REDUCTION (ASR)

Attack Surface Reduction (ASR) measures the effectiveness of a security method in minimizing the possible points of attack within a containerized environment. ASR can be expressed as a percentage reduction or the specific number of mitigated system calls. A significant reduction in attack surface indicates a more robust security posture, as fewer attack vectors remain available for exploitation. Figure 6 analysis assesses the ASR performance of various container security methods, providing insights into their strengths in reducing vulnerabilities and bolstering overall system security.

1) SUBSTANTIAL ATTACK SURFACE REDUCTION

Several methods demonstrate a substantial reduction in attack surface, reflecting a strong emphasis on minimizing vulnerabilities. μSWITCH [37] reports an ASR reduction ranging from 32.7% to 98.4%, showcasing a highly effective approach to reducing exploitable vectors. This significant reduction emphasizes the method's strong security focus, making it ideal for environments with high-security demands. Seccomp-eBPF [43] achieves a 55.4% reduction in the attack surface, effectively halving the potential attack vectors. This substantial improvement highlights the method's ability to enforce strict security policies while maintaining system integrity. These methods provide robust ASR performance, making them well-suited for environments where reducing the attack surface is paramount to preventing unauthorized access.

2) MODERATE TO SIGNIFICANT REDUCTION SOLUTIONS

Other methods achieve moderate to significant reductions in attack surface, contributing to overall system security but with varying degrees of impact. sysE [35] shows an ASR improvement between 7.3% and 21.5%, offering a scalable reduction in attack surface. This approach allows sysE to adapt to different environments with varying security needs. GyroidOS [54] records an 8.6% reduction, contributing to overall system hardening without dramatically altering the system's functionality. Prof-gen [57] achieves a 20.2% reduction in attack surface, indicating substantial progress in mitigating risks while maintaining system performance.

These solutions are effective for environments that balance reducing vulnerabilities and maintaining operational flexibility.

3) SECURITY FOCUS WITHOUT SPECIFIED METRICS

Several methods contribute to ASR through various mechanisms, including isolation, direct prevention, and system call limitations. Although their exact ASR metrics are not specified, their strategic focus on tightening security is clear: Bastion+ [45], PPASes [51], TCX [52], Lic-Sec [58], TZ-Container [59], BPFContain [60], and bpfbox [61] implement mechanisms such as enclave usage, isolation, and syscall restrictions to improve security. However, their specific impact on ASR needs to be quantified. These methods demonstrate strong security capabilities by focusing on preventing common attack vectors, making them valuable additions to security-conscious environments, even though exact ASR data is lacking.

4) DATA-DRIVEN AND EMPIRICAL REDUCTIONS

Some methods are described as quantified or empirical in terms of their ASR performance. PBRIEM [49] is noted for having a quantified approach to attack surface reduction, likely employing data-driven mechanisms to achieve measurable security improvements. HSCPA [53] shows an empirical decrease in ASR, suggesting that real-world data supports its effectiveness in reducing vulnerabilities. These methods highlight the importance of data-driven approaches in refining attack surface reduction strategies and validating their impact in operational environments.

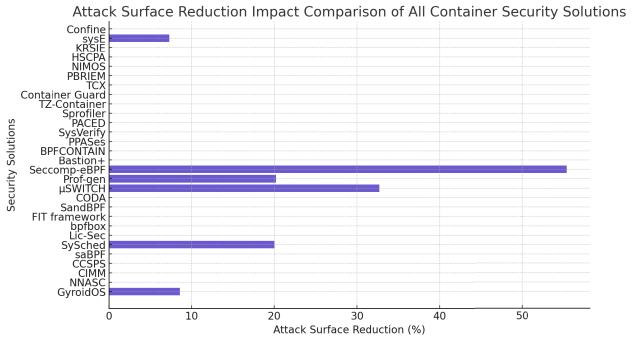

FIGURE 6. Attack surface reduction impact comparison of all container security solutions.

5) METHODS WITHOUT SPECIFIED ASR DATA

Several security solutions do not provide specific ASR metrics marked as Not Mentioned (NM). While these methods may still contribute to security, the lack of quantifiable data limits a complete understanding of their effectiveness in reducing the attack surface: Methods such as NIMOS [36], Confine [38], KRSIE [40], CODA [44], PACED [47], FIT framework [48], ContainerGuard [50], CCSPS [55], saBPF [16], and NNASC [62] lack specific ASR figures, leaving gaps in the understanding of their influence on reducing potential attack vectors. While these methods may offer robust security features, further disclosure of ASR data could enhance the evaluation of their effectiveness in reducing vulnerabilities.

6) POTENTIAL INCREASE IN ATTACK SURFACE

One method, SySched [42], shows a 20% increase in attack surface, likely due to the expansion of system functionality that introduces additional risk exposure. This increase highlights the potential trade-offs between enhancing functionality and maintaining a tight security perimeter. SySched's increased attack surface could pose a risk for environments prioritizing security over added functionality despite its other security features.

This analysis reveals the diversity in attack surface reduction (ASR) performance across container security solutions. Methods such as μSWITCH and Seccomp-eBPF demonstrate substantial reductions in attack surface, while other methods provide moderate improvements or focus on security features without quantifiable data. Decision-makers must weigh these ASR capabilities alongside the specific needs of their deployment environments, balancing security improvements with functionality and performance considerations.

E. DYNAMIC ADAPTABILITY (DA)

Dynamic Adaptability (DA) refers to a security solution's ability to dynamically adjust to new and evolving threats. In the rapidly changing landscape of technology, new vulnerabilities frequently emerge, making it essential for security methods to adapt in real-time. A technique with strong DA can evolve to address unforeseen threats, making it a critical feature in container security solutions. Figure 7 compares the dynamic adaptability of various container security methods, identifying those that can adjust to emerging threats and those that lack this capacity.

1) SOLUTIONS WITH DYNAMIC ADAPTABILITY

The following methods are designed with dynamic adaptability capabilities, indicated by a checkmark (✓). These methods can evolve in response to new threats, making them suitable for dynamic and fast-evolving security environments. sysE [35], NIMOS [36], SysVerify [39], and KRSIE [40] are equipped to handle evolving threats, making them highly adaptive and versatile in environments with a rapidly changing security landscape. SandBPF [41], Seccomp-eBPF [43], CODA [44], and Bastion+ [45] also possess robust DA features, allowing them to adjust to new vulnerabilities as they arise. PACED [47], FIT framework [48], ContainerGuard [50], HSCPA [53], and GyroidOS [54] are designed with flexibility in mind, ensuring they remain effective even as threat landscapes shift. CCSPS [55], Sprofiler [56], saBPF [16], BPFContain [60], bpfbox [61], and NNASC [62] are also built with DA in mind, making them highly reliable in environments that require continuous adaptation to security threats. These methods suit environments where agility and adaptability are critical, particularly in industries facing frequent security challenges or rapidly evolving technologies, such as cloud services or multi-tenant platforms.

2) SOLUTIONS LACKING DYNAMIC ADAPTABILITY

The following methods are marked with 'NAD' (Does Not Address Dynamic Adaptation), indicating that they may lack the capacity to adapt to new or changing threats dynamically: μSWITCH [37], Confine [38], SySched [42], CIMM [46], PPASes [51], TCX [52], Lic-Sec [58], and TZ-Container [59] do not address dynamic adaptability. While these methods offer robust security against known threats, their inability to adjust to new or evolving vulnerabilities may limit their effectiveness in dynamic security environments. Methods without DA may still provide solid protection against static, well-known threats. Still, their lack of adaptability could render them less effective in environments where new vulnerabilities are regularly discovered. As technology evolves, so do the attack vectors, making adaptability increasingly important.

3) LIMITED OR UNCLEAR DYNAMIC ADAPTABILITY

Specific methods need more information about their ability to handle dynamic threats, marked as 'PN' (Provided Null) or 'NAD'. PBRIEM [49] is marked with 'PN,' indicating that no information about its capability for dynamic adaptation is available. It is still being determined whether this solution can evolve with new threats. Prof-gen [57] is categorized as 'NAD,' meaning it does not currently address dynamic adaptation, which could limit its applicability in environments requiring flexible security measures. In environments where

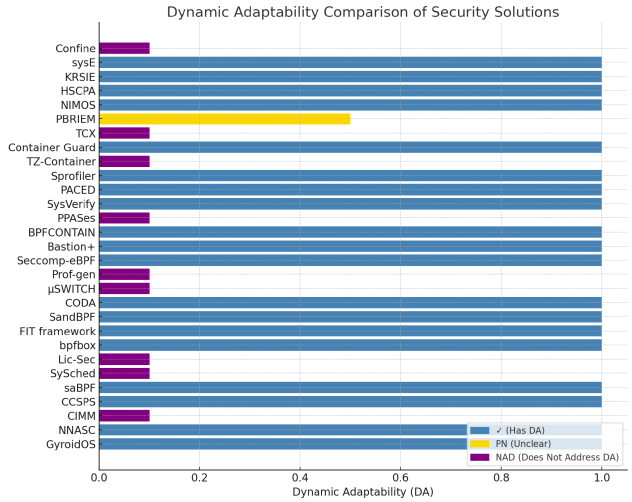

FIGURE 7. **Dynamic adaptability distribution of container security solutions.**

new threats emerge frequently, the absence of information or adaptability features may raise concerns about the long-term security efficacy of these methods.

Dynamic Adaptability (DA) is critical in container security solutions, particularly in fast-paced, rapidly evolving environments. Solutions like sysE, NIMOS, and Seccomp-eBPF are well-suited for such environments, providing the flexibility needed to adapt to emerging threats. In contrast, methods that do not address dynamic adaptability, such as μSWITCH and Lic-Sec, offer strong protection against known threats but may fall short in responding to new vulnerabilities. Organizations operating in highly dynamic environments should prioritize solutions with DA capabilities to ensure their systems remain secure against current and future threats. Those lacking DA may still provide robust protection but should be carefully evaluated for environments where security agility is less critical.

F. RESOURCE UTILIZATION (RU)

Resource Utilization (RU) measures the impact of container security solutions on system resources, such as CPU usage, memory overhead, and network bandwidth. Maintaining a low resource footprint ensures system efficiency and scalability, especially in resource-constrained environments. In this section, we evaluate the RU performance of various container security methods and highlight their suitability for different operational contexts.

1) EFFICIENT RESOURCE UTILIZATION SOLUTIONS

Some security methods are optimized for minimal resource consumption, making them ideal for limited resource availability. sysE [35] is highly optimized for efficient resource utilization, ensuring a low impact on system resources. This makes it a preferred choice for environments where performance efficiency is paramount. CODA [44] and CIMM [46] both demonstrate low resource usage, suggesting that they have minimal impact on CPU and memory performance, allowing systems to run securely without compromising on speed or responsiveness. PACED [47] is described as having a negligible footprint, making it one of the least intrusive solutions in terms of resource consumption.

These methods suit resource-constrained environments where maintaining system performance is critical alongside security.

2) MODERATE RESOURCE USAGE SOLUTIONS

Some methods balance security and resource consumption, making them suitable for environments that can tolerate moderate increases in resource usage. ContainerGuard [50] incurs a moderate resource increase of 4.50%, striking a balance between enhanced security and system performance. PPASes [51] records modest resource utilization, making it suitable for environments with moderate resource availability. TZ-Container [59] uses approximately 5% of system resources, which may be manageable depending on the deployment scenario. These methods are appropriate for environments where a moderate increase in resource usage is acceptable in exchange for robust security features.

3) HIGH RESOURCE CONSUMPTION SOLUTIONS

Some security methods impose substantial resource demands, potentially impacting performance in resource-constrained systems. μSWITCH [37] shows a significant increase in resource usage, possibly quadrupling consumption, which could present challenges for performance-sensitive environments. Seccomp-eBPF [43] registers a moderate rise in resource consumption, which could strain systems with limited resources, especially in high-density environments. TCX [52] requires 1GB of memory for secure containers within a virtualized environment, which may be significant depending on the available resources in the system. These methods are more suitable for security-centric environments where resource consumption can be justified in exchange for higher security.

4) NOTEWORTHY INSTANCES OF REDUCED RESOURCE USAGE

Specific methods stand out for their ability to reduce resource consumption, enhancing both security and efficiency. Prof-gen [57] distinguishes itself with a reported 20.2% reduction in resource usage, indicating improved efficiency alongside robust security. Lic-Sec [58] and GyroidOS [54] exhibit negligible resource utilization, making them highly efficient in system responsiveness. bpfbox [61] shows a slight reduction in resource consumption, hinting at an improvement in efficiency without sacrificing security. These methods are excellent choices for environments seeking to enhance security while optimizing resource usage.

5) UNSPECIFIED RESOURCE UTILIZATION

Several methods need to provide specific data on their resource utilization, indicated by 'NM' (Not Mentioned).

While these methods may offer robust security, more detailed RU data is needed to allow an understanding of their impact on system resources. Methods such as NIMOS [36], Confine [38], and KRSIE [40] have not specified their RU, leaving gaps in understanding their effect on system performance. PBRIEM [49] references 'cAdvisor' for monitoring resource usage but does not provide specific details, which suggests a reliance on external monitoring tools rather than reporting precise consumption metrics. These methods may still be valuable in particular contexts, but decision-makers may need further evaluation to determine their resource impact.

The Resource Utilization (RU) of container security methods varies widely, from highly efficient solutions like sysE, CODA, and PACED to resource-intensive methods such as μSWITCH and Seccomp-eBPF. When selecting a security solution, decision-makers must consider these factors against their specific environmental requirements and resource limitations. Environments with limited resources should prioritize efficiency-focused solutions, while security-centric environments may be able to tolerate higher resource consumption for the sake of enhanced protection.

G. COMPATIBILITY AND INTEGRATION WITH EXISTING PROTOCOLS

Integration with Protocols refers to how well a container security solution integrates with existing protocols and technologies, such as eBPF, Seccomp, Intel MPK, etc. Effective integration is critical for enhancing security without requiring significant modifications to existing infrastructure. A seamless integration ensures that security measures work harmoniously with the current system setup, improving performance and operational efficiency. Figure 8 compares the integration capabilities of various container security methods, assessing how well they align with current technologies and infrastructure.

1) SEAMLESS INTEGRATION SOLUTIONS

Several container security solutions are designed to seamlessly integrate with widely-used technologies, ensuring compatibility and efficient deployment. sysE [35] integrates with eBPF, leveraging the ability to execute programs directly in the Linux kernel, which is particularly advantageous for constrained computing environments. This integration allows sysE to provide security while maintaining high performance. Seccomp-eBPF [43] demonstrates advanced use of kernel privileges, reinforcing its strong cohesion with Linux's kernel security features. This makes it ideal for environments requiring close control over system calls and kernel-level operations. Bastion+ [45] integrates with AppArmor and SELinux, two of the most prominent Linux security modules. This multi-faceted approach allows Bastion+ to leverage the strengths of both systems, making it highly versatile in environments that require robust application-level and system-level security. SandBPF [41] works seamlessly with eBPF, reinforcing its role as a sophisticated tool for kernel-level operations and enabling it to provide efficient and secure monitoring at the kernel level. PBRIEM [49] utilizes Linux namespaces for process isolation, adhering to native Linux security capabilities. This integration allows it to create secure and isolated environments within containers, aligning with Linux's built-in isolation features.

These solutions seamlessly integrate with existing security frameworks, ensuring minimal disruption while enhancing security within current system infrastructures.

2) CUSTOM AND TAILORED INTEGRATION SOLUTIONS

Some methods require custom or tailored integration, making them more flexible but potentially more complex to implement in specific environments. μSWITCH [37] integrates with Intel MPK (Memory Protection Keys), which provides hardware-based protection for securing memory access. While this hardware-based integration is powerful, it may require a more specialized setup than software-only solutions. saBPF [16] uses custom integration methods that may require tailored solutions for specific system configurations. While this approach offers flexibility, it may also necessitate adjustments to the existing environment for optimal functionality. Lic-Sec [58] leverages AppArmor for enforcing application-centric security profiles, focusing on securing applications directly through customized security profiles. This method is particularly beneficial in environments requiring fine-grained control over application security. These methods, while powerful, may necessitate custom configurations or adjustments to fit within specific infrastructures, making them ideal for more complex or specialized environments.

3) AI-DRIVEN AND INNOVATIVE INTEGRATION

A few security methods are built on AI-driven technologies and innovative integration approaches, representing forward-thinking approaches to container security. NNASC [62] employs neural network analytics, an AI-driven method for threat detection. This approach is forward-looking and innovative, aiming to improve detection accuracy by analyzing system calls through machine learning models. While this is an advanced solution, it may require more infrastructure adjustments than traditional methods. bpfbox [61], with its permissive mode, allows for lenient policy application, making it suitable for auditing or transitional integration. This flexibility lets users gradually integrate security policies without imposing strict controls immediately. These solutions offer cutting-edge technology and machine learning-driven integration, making them valuable in environments where advanced threat detection and adaptability are priorities.

4) METHODS LACKING SPECIFIED INTEGRATION

Several methods do not specify their integration capabilities, indicated by 'NM' (Not Mentioned). While they may still offer robust security, their need for specified integration information limits their understanding of how they align with existing security frameworks. Solutions such as NIMOS [36],

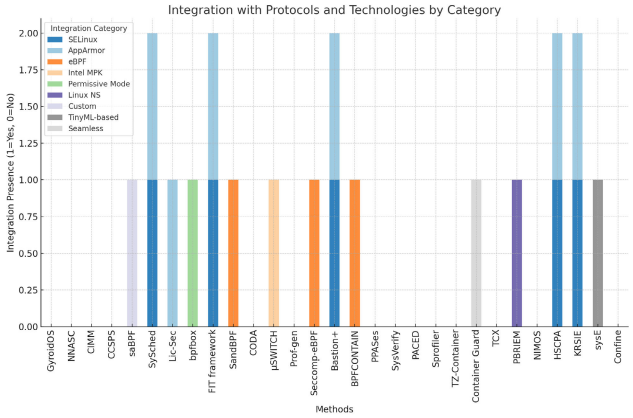

FIGURE 8. Integration with protocols and technologies by category.

Confine [38], SysVerify [39], CODA [44], CIMM [46], PACED [47], PPASes [51], TCX [52], GyroidOS [54], CCSPS [55], Prof-gen [57], and TZ-Container [59] do not provide details on their integration with protocols and technologies, leaving gaps in understanding how easily they can be deployed in existing infrastructures. For decision-makers, the lack of integration data may necessitate further investigation to assess whether these solutions can fit seamlessly within their current technology stack.

The integration capabilities of container security solutions vary widely. Some methods, such as sysE, Bastion+, and Seccomp-eBPF, offer seamless integration with existing technologies like eBPF and SELinux, making them ideal for environments that require efficient deployment without significant changes. Other solutions, like μSWITCH and NNASC, rely on custom integrations or advanced AI-driven methods, which may require more specialized configurations but offer powerful security enhancements. Decision-makers must weigh the integration capabilities of these methods against their current infrastructure to ensure a smooth deployment and effective security enhancement. Solutions that align well with existing protocols can help improve security without disrupting system operations, while those requiring more customization may offer advanced features but come with increased complexity.

H. TECHNOLOGIES EMPLOYED IN CONTAINER SECURITY SOLUTIONS

The Technologies employed by each container security solution provide insights into their effectiveness and strategic approach to security. Figure 9 presents a comparative analysis of the technologies employed by various container security solutions, ranging from traditional Linux kernel features to modern, AI-driven, and hardware-assisted innovations. eBPF emerges as the most widely adopted technology, utilized in 16 solutions, accounting for approximately 43% of the total, underscoring its importance for kernel-level monitoring and flexible operations. Seccomp ranks second, with adoption in three solutions, representing about 8%, focusing on restricting system calls to minimize attack surfaces. Dynamic and Static Analysis and Static Analysis collectively account for approximately 22%, reflecting their significance in lifecycle security through vulnerability detection and runtime behavior monitoring. Other technologies, such as Process Namespaces, Hardware-Assisted Methods (e.g., AMD SEV, ARM TrustZone), and AI/ML-based Approaches, each make up smaller proportions, ranging from 2-3%, but demonstrate emerging trends in addressing specific security challenges. The diversity of technologies highlights the variety of strategies employed in modern container security to tackle evolving threat landscapes. By understanding the technological foundations of these methods, organizations can better align their security strategies with their infrastructure capabilities and performance requirements.

1) EXTENDED BERKELEY PACKET FILTER (eBPF) INTEGRATION

eBPF is a widely adopted technology for high-performance, flexible packet filtering and process monitoring. Several container security methods leverage eBPF for its robust kernel-level operations: sysE [35], KRSIE [40], SandBPF [41], Seccomp-eBPF [43], CODA [44], Bastion+ [45], CIMM [46], CCSPS [55], Sprofiler [56], saBPF [16], and BPFContain [60] all integrate eBPF, demonstrating a trend toward using this technology for efficient packet filtering and flexible process monitoring. The widespread use of eBPF reflects its importance in modern container security. It provides real-time visibility into system calls and enhances performance without significantly increasing resource usage.

2) SECCOMP-BASED SOLUTIONS

Seccomp is a Linux kernel feature that restricts the system calls available to a process, reducing the attack surface by limiting the number of possible exploits. It is known for its simplicity and effectiveness in container security: NIMOS [36], SySched [42], and GyroidOS [54] all rely on Seccomp to limit system call access, reinforcing the container's security by reducing the number of exposed system calls. Seccomp provides an essential layer of protection for containers by preventing unnecessary system calls, which could be exploited in attacks, making it a low-overhead but highly effective solution.

3) MULTIFACETED TECHNOLOGY STACKS

Some solutions integrate multifaceted technology stacks that combine various technologies for robust, layered security: μSWITCH [37] combines Intel®Memory Protection Keys (MPK), WebAssembly (Wasm), and Process Isolation (PI). This robust technology stack provides multiple layers of security, combining hardware-based protection (MPK) with software isolation (Wasm), making μSWITCH ideal for complex, high-security environments. These multifaceted approaches offer hardware and software-based protections, creating strong defenses that are difficult to bypass.

4) STATIC AND DYNAMIC ANALYSIS SOLUTIONS

Solutions that use static and dynamic analysis provide comprehensive security by examining container behavior before runtime and monitoring activity during execution. Confine [38], SysVerify [39], and HSCPA [53] employ both static and dynamic analysis to ensure security throughout the container lifecycle. This approach allows these methods to detect potential vulnerabilities before deployment and monitor containers in real time to catch anomalous behavior. Prof-gen [57] combines syscall traces with static analysis, providing an additional layer of security by analyzing system call patterns for anomalies and weaknesses. These solutions are excellent for environments requiring continuous monitoring and pre-runtime vulnerability assessments.

5) AI-DRIVEN AND MACHINE LEARNING TECHNOLOGIES

Several modern container security methods incorporate AI-driven and machine-learning techniques to enhance threat detection and response. ContainerGuard [50] implements Variational Autoencoders (VAEs), an advanced machine-learning technique for anomaly detection. This AI-driven approach helps ContainerGuard adapt to new threats by learning normal and abnormal behavior patterns. NNASC [62] uses neural network (NN) system call analysis, providing an advanced, machine-learning-based method for detecting abnormal behavior indicative of security breaches. AI-based solutions like these are especially suited for environments where threat landscapes are dynamic, offering adaptive security that can evolve with emerging threats.

6) HARDWARE-BASED SECURITY FEATURES

Hardware-based security is becoming increasingly popular as it provides strong isolation and protection at the physical level, offering enhanced security for high-risk environments. PPASes [51] integrates Arm hardware features, Intel Software Guard Extensions (SGX), and TrustZone, providing hardware-based isolation and secure execution environments. TCX [52] employs AMD Secure Encrypted Virtualization (SEV), which offers encrypted virtual machines, ensuring that data and processes remain secure even if the hypervisor is compromised. These hardware-assisted methods are ideal for environments that require physical solid isolation and protection, such as financial services or government systems.

7) SPECIFICATION-DRIVEN AND NAMESPACE-BASED SOLUTIONS

Specific methods employ specification-driven technologies or process namespaces for securing container environments. The FIT framework [48] utilizes spec-based technologies, likely involving formal verification or adherence to predefined security specifications. This approach helps ensure compliance with security standards and protocols, reducing the risk of vulnerabilities. PBRIEM [49] refers to Linux namespaces, a fundamental container isolation technology that allows for process separation and resource control within the container, further reinforcing container security. These solutions focus on specification compliance and process isolation, which are essential for environments that must adhere to strict security regulations.

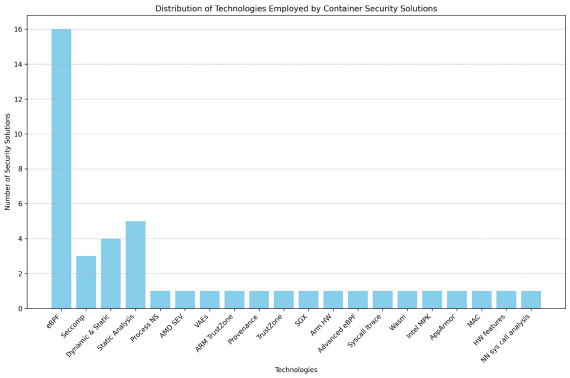

FIGURE 9. Distribution of technologies employed by container security solutions.

8) MANDATORY ACCESS CONTROL (MAC) AND POLICY ENFORCEMENT

Several solutions employ Mandatory Access Control (MAC) to enforce strict security policies: Lic-Sec [58] uses MAC alongside AppArmor, ensuring strict enforcement of administrator-defined security policies. This method is particularly effective for application-level security where fine-grained access control is required. MAC-based solutions are suitable for environments where administrator-defined security rules must be enforced to limit access to critical system resources.

The variety of technologies employed by container security methods showcases the diverse range of approaches available to organizations. From traditional Linux kernel features like Seccomp and AppArmor to advanced AI-driven and hardware-assisted methods, the choice of technology can significantly impact the effectiveness and suitability of a security solution. Organizations must assess their performance needs, infrastructure capabilities, and threat models to select the most appropriate security solutions. Whether leveraging the power of eBPF for flexible monitoring or using machine learning for adaptive threat detection, the right combination of technologies can significantly enhance container security while maintaining system efficiency.

In analyzing the extensive review, it's clear that the domain of container security offers a diverse range of solutions, each with its strengths and limitations. Organizations must carefully consider several key factors when navigating this complex landscape. These include the performance impact, the types of attacks addressed, the accuracy of detection (measured by false positives and negatives), the reduction of the attack surface, and adaptability to emerging threats. Additionally, the impact on system resources and compatibility with existing protocols and technologies are crucial, affecting integration and operational efficiency. These solutions employ various technologies, from

innovative uses of eBPF and machine learning to established Linux security modules, showcasing a rapidly evolving field seeking a balance between strong security and system performance. As the review emphasizes, stakeholders must assess these aspects collectively to ensure a robust security stance is responsive to current threats and adaptable to future security challenges. The scalability of containerization security solutions can be judged by how well they adapt to changes in containerized applications and how they scale with the infrastructure. Solutions like sysE [35] and SysVerify [39] are highly adaptable, employing technologies like eBPF, which offer a modern approach to security. On the other end, solutions like μSWITCH [37] and TCX [52] focus on specialized security improvements but may require additional resource allocations or lack in dynamic adaptability. When selecting an appropriate security solution, one must balance the desired level of security against the acceptable degree of performance overhead and the need for adaptability to ever-changing security threats.

VI. OPEN RESEARCH ISSUES AND CHALLENGES

The advent of containerization has transformed how businesses deploy and manage applications. However, challenges arising from common kernel architectures require further research and development. This paper extensively surveys the current state-of-the-art security solutions, capturing technical details and examining open research issues and challenges. Figure 10 illustrates these open research issues and challenges.

A. COMPREHENSIVE ATTACK SURFACE MANAGEMENT

In shared kernel architectures, containers share a common kernel while isolated, ensuring separation. This setup inherently increases the complexity of the attack surface. Existing security measures often focus on specific layers and components, such as the container runtime and orchestration layer. Still, they are only effective for some parts of the stack, leaving gaps in the system's protection. Solutions like SandBPF [41], SySched [42], TCX [52], FIT framework [48], CCSPS [55], Sprofiler [56], TZ-Container [59], and Seccomp-eBPF [43] address particular attack vectors but do not cover all possible attack vectors in the system. As attacks evolve to bypass these layers individually, it is crucial to strengthen all layers, including the container runtime, orchestrator, and kernel. Integrated, dynamic security frameworks for the multidimensional attack surfaces of containerized environments should be developed to meet this imperative need. Future research should focus on creating security solutions that ensure protection across all aspects of containers, from images and registries to the core infrastructure.

B. OPTIMIZING SECURITY-PERFORMANCE TRADE-OFF

Many times security mechanisms create congestion on the system. For example, the massive use of monitoring and logging slows the application performance, which is critical in high-performance systems. The trade-off between optimal security and minimal performance degradation is vital to be able to gain wider acceptance of container technologies in industries where the performance is critical. Existing techniques like SysVerify [39], PACED, Sprofiler [56], KRSIE [40], sysE, μSWITCH [37] and Seccomp-eBPF [43] demonstrate either high performance or security, but seldom both. Research should be directed at the junction of high performance and high security so that container technologies can be widely accepted. Research should be conducted to discover effective ways of eliminating security procedures' influence on productivity. This can be done by creating lighter and more effective security protocols and using machine learning for advanced threat detection.

C. SECURITY FOR DYNAMIC AND SCALABLE ENVIRONMENTS

Containers are considered dynamic in nature, as instances are frequently created and destroyed. Traditional security models that take a static and less dynamic environment as their given will be less effective in such environments. On the contrary, we already have SySVerify [39] and KRSIE [40], which are able to change and adjust dynamically as the needs arise, but the architecture and approach of those solutions that are scalable and reactive are having a problem coping with real-time security adjustments. A modifiable security solution is required considering the dynamic nature of containerized settings that adjust its technology as well. This includes immediate security architectures as well as methods that can be scaled without necessarily being connected with the dynamism of container workloads in different contexts over a period of time.

D. ENHANCED ISOLATION TECHNIQUES

While namespaces and control groups offer basic isolation, their reliance on the underlying kernel may not be sufficient for high-risk environments. Namespace isolation, while effective to a certain extent, can still pose potential leaks and flow issues. The goal of mechanisms like cgroups is to control resource use, but their configurations can be intricate and challenging to manage. Similarly, maintaining seccomp profiles, which restrict the system calls a container can make, can be complex and reactive. The state-of-the-art examples like BPFContain [60] and TZ-Container [59] guarantee isolation of processes and memory. A fine isolation mechanism is necessary to prevent partial break-outs that may compromise several containers or even the whole system, especially when containers are built using responsive kernel architecture where they share a common kernel. Therefore, researchers should focus on improving isolation techniques, for instance, through new virtualization technologies or hardware-assisted solutions that offer better security without high overheads.

E. HANDLING EMERGING THREATS

It is a well-known fact that in today's ever-changing cybersecurity environment, we see the emergence of new

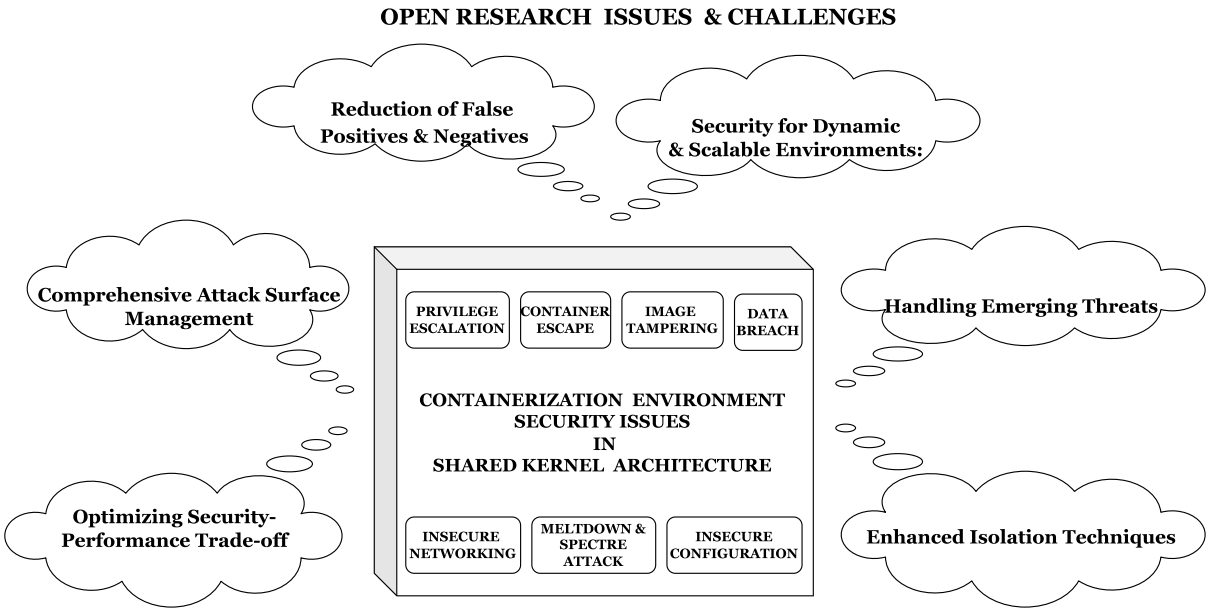

FIGURE 10. Open research issues and challenges.

threats and advanced attack methods that try to exploit existing weaknesses, especially in vibrant container environments. While we are currently relying on static as well as dynamic defenses like these in the form of HSCPA [53] and GyroidOS [54], the ever-increasing complexity of threats, along with the rate of change in the field encourage the development of novel detection techniques coupled with new security practices that will help the industry detect and quickly adapt to new vulnerabilities. The continuous study is paramount for fighting ever-changing ones and for reinforcing defense systems. This entails the crafting of threat detection algorithms, improving threat intelligence, and employing analysis, anomaly detection, and deception techniques to mitigate advanced attacks effectively. It is paramount that security mechanisms are agile enough to adapt to evolving discoveries of vulnerabilities. Discussing the security mechanisms resistant to evasion, behavioral analysis, anomaly detection, and deception helps us keep ourselves safer from complex attacks.

F. REDUCTION OF FALSE POSITIVES AND NEGATIVES

The unreliability of alerts due to false positives and negatives can cause real problems with the productivity of security operations that deal with containerized environments. Waste of resources takes place when false positives are investigated. However, the false negatives signal missed detections of the real threats, disallowing the cases and possible security breaches to be worked out. Therefore, we should create and improve the detection mechanisms that minimize false alarms and missed detections. Increased algorithms and models are necessary, and the program can be made to understand the varied patterns of legitimate and malicious activities carried out within the containers to determine actual threats from benign anomalies with more accuracy. For example, the anomaly detection mechanisms in NNASC [62] and the syscall analysis used by bpfbox [61] are examples of current techniques that display progress in the way different behavior is exhibited and have the possibility of differentiating between normal and suspicious behavior. Nevertheless, these techniques could be made more potent by using machine learning and artificial intelligence, which can be used to analyze large datasets and provide precise learning outcomes. On top of that, using threat intelligence and bringing contextual information to the picture will decrease the number of false alarms in the threat detection systems.

Containers' security in shared kernel environments also involves many problems, which in turn leads to the necessity of dedicated research aimed at achieving a balance between effectiveness and efficiency in a problem's solution. The fact is that when container technology is continually developing, the complexity of the security landscape similarly changes. These loopholes need to be identified and bypassed for these containerized systems to progress in their security capabilities as their applications grow in different computing environments. The primary emphasis in future research must be on developing adaptable, integrated, and performance-oriented security systems that will stand up against the dynamic and multifaceted characteristics of modern containerized infrastructures.

VII. CONCLUSION

This paper presents a unique systematic review of the current landscape of security challenges inherent in containerization

technologies within shared kernel environments. We have explored an array of state-of-the-art security solutions, addressing the multifaceted risks associated with container deployment, particularly in cloud computing frameworks. Through a detailed thematic taxonomy, we have categorized existing literature on various security mechanisms, highlighting their functionalities and contributions to enhancing container security. Our comparative analysis discloses that while numerous innovative security strategies have been proposed, several open areas remain demanding further research. Firstly, despite significant advancements in isolation techniques and kernel security, continuous efforts are necessary to refine these solutions to better guard against attack vectors. Secondly, the integration of security practices into DevOps pipelines poses ongoing challenges, suggesting a need for more dynamic and automated security tools that can seamlessly adapt to rapid deployment cycles. Moreover, with the increasing adoption of containers in high-risk environments, there is an urgent need to develop more robust methods for real-time security monitoring and incident response. Additionally, the rise of microservices architectures introduces complexity in managing security policies, requiring more granular and context-aware security mechanisms. Furthermore, it is imperative to address the scalability of security solutions to keep pace with the growth of containerized applications across diverse environments. Future research should also focus on enhancing the security of container orchestration tools, which play a pivotal role in managing container lifecycles but are often susceptible to targeted exploits. As discussed above, while the field of container security has made notable progress, significant challenges remain. Future work should aim to devise comprehensive, intelligent security frameworks that can anticipate and mitigate emerging threats in containerized systems within a shared kernel environment. This includes extending research to cover emerging paradigms such as serverless computing, which could integrate with or replace traditional container architectures, presenting new security dynamics.

ACKNOWLEDGMENT

The authors acknowledge the invaluable technical support provided by the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. Additionally, they extend their appreciation to the anonymous reviewers for their insightful comments and constructive feedback, which have greatly contributed to improving the quality and clarity of this article.

CONFLICT OF INTEREST

The authors declare that there are no conflicts of interest regarding the publication of this paper. No financial, personal, or other relationships with organizations or individuals have influenced the content or findings presented in this manuscript.

REFERENCES

[1] L. Camiciola, "Cloudifying desktop applications: Your laptop is your data center," Ph.D. thesis, Dept. Comput. Eng., Polytech. Univ. Turin, Italy, 2021.

[2] E. Casalicchio and S. Iannucci, "The state-of-the-art in container technologies: Application, orchestration and security," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 17, p. e5668, Sep. 2020.

[3] A. Solis, W. J. Allen, and E. Ferlanti, "Containerizing visualization software: Experiences and best practices," in *Proc. Pract. Exper. Adv. Res. Comput.*, Jul. 2022, pp. 1–8.

[4] Z. Zhong and R. Buyya, "A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources," *ACM Trans. Internet Technol. (TOIT)*, vol. 20, no. 2, pp. 1–24, May 2020.

[5] T. Siddiqui, S. A. Siddiqui, and N. A. Khan, "Comprehensive analysis of container technology," in *Proc. 4th Int. Conf. Inf. Syst. Comput. Netw. (ISCON)*, Nov. 2019, pp. 218–223.

[6] N. Akhilesh, M. Aniruddha, A. Ghosh, and K. Sindhu, "A system to create automated development environments using Docker," in *Proc. 8th Innov. Comput. Sci. Eng. (ICICSE)*. Cham, Switzerland: Springer, 2021, pp. 555–563.

[7] H. J. Syed, A. Gani, R. W. Ahmad, M. K. Khan, and A. I. A. Ahmed, "Cloud monitoring: A review, taxonomy, and open research issues," *J. Netw. Comput. Appl.*, vol. 98, pp. 11–26, Nov. 2017.

[8] M. Woudstra, "Designing a container management solution to improve flexibility and portability, and reducing cost for ipaas solutions," Master's thesis, Dept. Elect. Eng., Math. Comput. Sci., Univ. Twente, The Netherlands, 2022.

[9] O. Bentaleb, A. S. Z. Belloum, A. Sebaa, and A. El-Maouhab, "Containerization technologies: Taxonomies, applications and challenges," *J. Supercomput.*, vol. 78, no. 1, pp. 1144–1181, Jan. 2022.

[10] A. Kumar and M. Agarwal, "Quick service during DDoS attacks in the container-based cloud environment," *J. Netw. Comput. Appl.*, vol. 229, Sep. 2024, Art. no. 103946.

[11] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, "Emerging trends, techniques and open issues of containerization: A review," *IEEE Access*, vol. 7, pp. 152443–152472, 2019.

[12] M. Bélair, S. Laniepce, and J.-M. Menaud, "Leveraging kernel security mechanisms to improve container security: A survey," in *Proc. 14th Int. Conf. Availability, Rel. Secur.*, Aug. 2019, pp. 1–6.

[13] A. S. Thyagaturu, P. Shantharama, A. Nasrallah, and M. Reisslein, "Operating systems and hypervisors for network functions: A survey of enabling technologies and research studies," *IEEE Access*, vol. 10, pp. 79825–79873, 2022.

[14] I. Mavridis and H. Karatza, "Combining containers and virtual machines to enhance isolation and extend functionality on cloud computing," *Future Gener. Comput. Syst.*, vol. 94, pp. 674–696, May 2019.

[15] A. Bhardwaj and C. R. Krishna, "Virtualization in cloud computing: Moving from hypervisor to containerization—A survey," *Arabian J. Sci. Eng.*, vol. 46, no. 9, pp. 8585–8601, Sep. 2021.

[16] S. Y. Lim, B. Stelea, X. Han, and T. Pasquier, "Secure namespaced kernel audit for containers," in *Proc. ACM Symp. Cloud Comput.*, Nov. 2021, pp. 518–532.

[17] I. Odun-Ayo, O. Ajayi, and C. Okereke, "Virtualization in cloud computing: Developments and trends," in *Proc. Int. Conf. Next Gener. Comput. Inf. Syst. (ICNGCIS)*, Dec. 2017, pp. 24–28.

[18] V. S. D. Priya, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Comput. Secur.*, vol. 135, Dec. 2023, Art. no. 103490.

[19] D. Zhan, Z. Yu, X. Yu, H. Zhang, L. Ye, and L. Liu, "Securing operating systems through fine-grained kernel access limitation for IoT systems," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5378–5392, Mar. 2023.

[20] M. Cinque, R. Della Corte, and A. Pecchia, "Micro2vec: Anomaly detection in microservices systems by mining numeric representations of computer logs," *J. Netw. Comput. Appl.*, vol. 208, Dec. 2022, Art. no. 103515.

[21] T. Yamauchi, Y. Akao, R. Yoshitani, Y. Nakamura, and M. Hashimoto, "Additional kernel observer: Privilege escalation attack prevention mechanism focusing on system call privilege changes," *Int. J. Inf. Secur.*, vol. 20, no. 4, pp. 461–473, Aug. 2021.

[22] N. Yang, W. Shen, J. Li, Y. Yang, K. Lu, J. Xiao, T. Zhou, C. Qin, W. Yu, J. Ma, and K. Ren, "Demons in the shared kernel: Abstract resource attacks against OS-level virtualization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 764–778.

[23] A. Yi Wong, E. Getahun Chekole, M. Ochoa, and J. Zhou, "Threat modeling and security analysis of containers: A survey," 2021, *arXiv:2111.11475*.
[24] S. M. Magdaleno, V. M. Rocha, and R. Parra, "A review of security risks and countermeasures in containers," *Int. J. Secur. Netw.*, vol. 16, no. 3, p. 183, 2021.
[25] M. S. Oluyede, J. Mart, A. Olusola, and G. Olatuja, "Container security in cloud environments," *ScienceOpen Preprints*, pp. 1–24, Feb. 2024.
[26] P. Horchulhack, E. K. Viegas, A. O. Santin, F. V. Ramos, and P. Tedeschi, "Detection of quality of service degradation on multi-tenant containerized services," *J. Netw. Comput. Appl.*, vol. 224, Apr. 2024, Art. no. 103839.
[27] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019.
[28] F. Minna and F. Massacci, "SoK: Run-time security for cloud microservices. Are we there yet?" *Comput. Secur.*, vol. 127, Apr. 2023, Art. no. 103119.
[29] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the security of containers: Threat modeling, attack analysis, and mitigation strategies," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103140.
[30] Y. Kawanishi, H. Nishihara, H. Yoshida, H. Yamamoto, and H. Inoue, "A study on threat analysis and risk assessment based on the 'Asset container' method and CWSS," *IEEE Access*, vol. 11, pp. 18148–18156, 2023.
[31] H. Zhu, C. Gehrmann, and P. Roth, "Access security policy generation for containers as a cloud service," *Social Netw. Comput. Sci.*, vol. 4, no. 6, p. 748, Sep. 2023.
[32] O. Javed and S. Toor, "Understanding the quality of container security vulnerability detection tools," 2021, *arXiv:2101.03844*.
[33] M. Compastié, R. Badonnel, O. Festor, and R. He, "From virtualization security issues to cloud protection opportunities: An in-depth analysis of system virtualization models," *Comput. Secur.*, vol. 97, Oct. 2020, Art. no. 101905.
[34] K. V. Netaji and G. P. Bhole, "A comprehensive survey on container resource allocation approaches in cloud computing: State-of-the-art and research challenges," in *Proc. Web Intell.*, vol. 19. Amsterdam, The Netherlands: IOS Press, 2021, pp. 295–316.
[35] T. Zheng, Y. Qiu, Y. Zheng, Q. Wang, and X. Chen, "Enhancing TinyML-based container escape detectors with systemcall semantic association in UAVs networks," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21158–21169, Jun. 2024.
[36] S. Song, S. Suneja, M. V. Le, and B. Tak, "On the value of sequence-based system call filtering for container security," in *Proc. IEEE 16th Int. Conf. Cloud Comput. (CLOUD)*, Daegu, South Korea, Jul. 2023, pp. 296–307.
[37] D. Peng, C. Liu, T. Palit, P. Fonseca, A. Vahldiek-Oberwagner, and M. Vij, "μSwitch: Fast kernel context isolation with implicit context switches," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2956–2973.
[38] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, "Confine: Fine-grained system call filtering for container attack surface reduction," *Comput. Secur.*, vol. 132, Sep. 2023, Art. no. 103325.
[39] D. Zhan, Z. Yu, X. Yu, H. Zhang, and L. Ye, "Shrinking the kernel attack surface through static and dynamic syscall limitation," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1431–1443, Mar. 2023.
[40] S. Song, S. Suneja, M. V. Le, and B. Tak, "Sequence-based system call filtering for enhanced container security, is it beneficial?" in *Proc. IEEE/ACM 23rd Int. Symp. Cluster, Cloud Internet Comput. Workshops (CCGridW)*, Bangalore, India, May 2023, pp. 278–280.
[41] S. Y. Lim, X. Han, and T. Pasquier, "Unleashing unprivileged eBPF potential with dynamic sandboxing," in *Proc. 1st Workshop eBPF Kernel Extensions*, New York, NY, USA. ACM, Sep. 2023, pp. 42–48.
[42] M. V. Le, S. Ahmed, D. Williams, and H. Jamjoom, "Securing container-based clouds with syscall-aware scheduling," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, Jul. 2023, pp. 812–826.
[43] J. Jia, Y. Zhu, D. Williams, A. Arcangeli, C. Canella, H. Franke, T. Feldman-Fitzthum, D. Skarlatos, D. Gruss, and T. Xu, "Programmable system call security with eBPF," 2023, *arXiv:2302.10366*.
[44] M. Zhan, Y. Li, H. Yang, G. Yu, B. Li, and W. Wang, "Coda: Runtime detection of application-layer CPU-exhaustion DoS attacks in containers," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1686–1697, May 2022.
[45] J. Nam, S. Lee, P. Porras, V. Yegneswaran, and S. Shin, "Secure inter-container communications using XDP/eBPF," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 934–947, Apr. 2023.
[46] N. Yang, C. Chen, T. Yuan, Y. Wang, X. Gu, and D. Yang, "Security hardening solution for Docker container," in *Proc. Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discovery (CyberC)*, Suzhou, China, Oct. 2022, pp. 252–257.
[47] M. Abbas, S. Khan, A. Monum, F. Zaffar, R. Tahir, D. Eyers, H. Irshad, A. Gehani, V. Yegneswaran, and T. Pasquier, "PACED: Provenance-based automated container escape detection," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Pacific Grove, CA, USA, Sep. 2022, pp. 261–272.
[48] T. Madi and P. Esteves-Verissimo, "A fault and intrusion tolerance framework for containerized environments: A specification-based error detection approach," in *Proc. Int. Workshop Secure Reliable Microservices Containers (SRMC)*, Vienna, Austria, Sep. 2022, pp. 1–8.
[49] K. Wu, D. Yang, X. Gao, W. Yang, M. Li, and D. Wang, "Process based container escape monitoring and resource isolation scheme," in *Proc. 13th Int. Conf. Inf. Commun. Syst. (ICICS)*, Irbid, Jordan, Jun. 2022, pp. 54–58.
[50] Y. Wang, Q. Wang, X. Chen, D. Chen, X. Fang, M. Yin, and N. Zhang, "ContainerGuard: A real-time attack detection system in container-based big data platform," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3327–3336, May 2022.
[51] A. Van't Hof and J. Nieh, "BlackBox: A container security monitor for protecting containers on untrusted operating systems," in *Proc. 16th USENIX Symp. Operating Syst. Design Implement. (OSDI 22)*, 2022, pp. 683–700.
[52] F. Brasser, P. Jauernig, F. Pustelnik, A.-R. Sadeghi, and E. Stapf, "Trusted container extensions for container-based confidential computing," 2022, *arXiv:2205.05747*.
[53] Y. Xing, X. Wang, S. Torabi, Z. Zhang, L. Lei, and K. Sun, "A hybrid system call profiling approach for container protection," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1068–1083, Jun. 2023.
[54] F. Wruck, V. Sarafov, F. Jakobsmeier, and M. Weiß, "GyroidOS: Packaging Linux with a minimal surface," in *Proc. ACM Workshop Secure Trustworthy Cyber-Phys. Syst.*, Baltimore, MD, USA. ACM, Apr. 2022, pp. 87–96.
[55] Z. Guo, Z. Lv, N. Li, T. Yuan, X. Gao, and Z. Yuan, "Comprehensive defense scheme against container escape related to container management procedure," in *Proc. Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discovery (CyberC)*, Oct. 2022, pp. 263–266.
[56] T. Iiguni, H. Kamei, and K. Saisho, "Sprofiler: Automatic generating system of container-native system call filtering rules for attack surface reduction," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Las Vegas, NV, USA, Dec. 2021, pp. 704–709.
[57] S. Kim, B. J. Kim, and D. H. Lee, "Prof-gen: Practical study on system call whitelist generation for container attack surface reduction," in *Proc. IEEE 14th Int. Conf. Cloud Comput. (CLOUD)*, Chicago, IL, USA, Sep. 2021, pp. 278–287.
[58] H. Zhu and C. Gehrmann, "Lic-Sec: An enhanced AppArmor Docker security profile generator," *J. Inf. Secur. Appl.*, vol. 61, Sep. 2021, Art. no. 102924.
[59] Z. Hua, Y. Yu, J. Gu, Y. Xia, H. Chen, and B. Zang, "TZ-container: Protecting container from untrusted OS with ARM TrustZone," *Sci. China Inf. Sci.*, vol. 64, no. 9, Sep. 2021, Art. no. 192101.
[60] W. Findlay, D. Barrera, and A. Somayaji, "BPFContain: Fixing the soft underbelly of container security," 2021, *arXiv:2102.06972*.
[61] W. Findlay, A. Somayaji, and D. Barrera, "Bpfbox: Simple precise process confinement with eBPF," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*. New York, NY, USA: ACM, Nov. 2020, pp. 91–103.
[62] H. Gantikow, T. Zöhner, and C. Reich, "Container anomaly detection using neural networks analyzing system calls," in *Proc. 28th Euromicro Int. Conf. Parallel, Distrib. Netw.-Based Process. (PDP)*, Mar. 2020, pp. 408–412.
[63] M. S. Rahaman, A. Islam, T. Cerny, and S. Hutton, "Static-analysis-based solutions to security challenges in cloud-native systems: Systematic mapping study," *Sensors*, vol. 23, no. 4, p. 1755, Feb. 2023.
[64] S. L. Rocha, F. L. Lopes de Mendonca, R. S. Puttini, R. R. Nunes, and G. D. A. Nze, "DCIDS—Distributed container IDS," *Appl. Sci.*, vol. 13, no. 16, p. 9301, Aug. 2023.
[65] S. Lee and J. Nam, "Kunerva: Automated network policy discovery framework for containers," *IEEE Access*, vol. 11, pp. 95616–95631, 2023.
[66] M. Stolet, L. Arzola, S. Peter, and A. Kaufmann, "Virtuoso: High resource utilization and μs-scale performance isolation in a shared virtual machine TCP network stack," 2023, *arXiv:2309.14016*.
[67] J. Flora, M. Teixeira, and N. Antunes, "μDetector: Automated intrusion detection for microservices," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Mar. 2023, pp. 748–752.

[68] C. Cao, A. Blaise, S. Verwer, and F. Rebecchi, "Learning state machines to monitor and detect anomalies on a kubernetes cluster," in *Proc. 17th Int. Conf. Availability, Rel. Secur.*, Aug. 2022, pp. 1–9.

[69] N. Pecka, L. Ben Othmane, and A. Valani, "Privilege escalation attack scenarios on the DevOps pipeline within a kubernetes environment," in *Proc. Int. Conf. Softw. Syst. Processes Int. Conf. Global Softw. Eng.*, Pittsburgh, PA, USA, May 2022, pp. 45–49.

[70] D. Reti and N. Becker, "Escape the fake: Introducing simulated container-escapes for honeypots," 2021, *arXiv:2104.03651*.

[71] M. Reeves, D. J. Tian, A. Bianchi, and Z. B. Celik, "Towards improving container security by preventing runtime escapes," in *Proc. IEEE Secure Develop. Conf. (SecDev)*, Atlanta, GA, USA, Oct. 2021, pp. 38–46.

[72] H. Gao, L. Li, H. Lin, J. Wan, D. Deng, and J. Li, "DECH: A novel attack pattern of cloud environment and its countermeasures," in *Proc. IEEE 5th Int. Conf. Cryptography, Secur. Privacy (CSP)*, Jan. 2021, pp. 117–122.

[73] S. Shringarputale, P. McDaniel, K. Butler, and T. La Porta, "Co-residency attacks on containers are real," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, New York, NY, USA: ACM, Nov. 2020, pp. 53–66.

[74] A. F. Baarzi, G. Kesidis, D. Fleck, and A. Stavrou, "Microservices made attack-resilient using unsupervised service fissioning," in *Proc. 13th Eur. Workshop Syst. Secur.*, Heraklion, Greece, Apr. 2020, pp. 31–36.

[75] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on Linux container security: Attacks and countermeasures," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, San Juan, PR, USA, Dec. 2018, pp. 418–429.

SEHAR ZEHRA received the Master of Science degree in computer science from the Sir Syed University of Engineering and Technology, in 2012. She is currently pursuing the Ph.D. degree in computer science with the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. She has amassed 19 years of teaching experience and is a full-time Assistant Professor with the Khursheed Government Girls Degree College, Karachi. Over her extensive teaching career, she has contributed significantly to her students' academic and professional growth, supervising numerous projects, and teaching various courses across the curriculum.

HASSAN JAMIL SYED received the bachelor's degree in electrical engineering from the Quaid-e-Awam University of Engineering, Sciences and Technology, Nawabshah, Pakistan, in 2003, the master's degree in personal mobile and satellite communication from the University of Bradford, England, U.K., in 2008, and the Ph.D. degree in computer science from the University of Malaya, Malaysia, in 2019. After completing his Ph.D., he was an Assistant Professor with the Department of Computer Sciences, National University of Computer and Emerging Sciences, Karachi, Pakistan, from August 2019 to June 2022. He then was a Senior Lecturer with the Faculty of Computing and Informatics, Universiti Malaysia Sabah (UMS), Sabah, Malaysia, for two years. He also worked with WorldCall Telecom Ltd., Karachi, as a Network Engineer, for two years, and with the Faculty of Engineering Science and Technology, Iqra University, Karachi, for four years. He is currently a Senior Lecturer with the School of Technology, Asia Pacific University of Innovation and Technology (APU), Kuala Lumpur, Malaysia. Throughout his career, he has supervised several Ph.D./M.S. thesis/final year projects and taught courses in the electronics, telecommunication, and computer science departments. His research interests include cloud computing, cybersecurity, SDN, NFV, eBPF, and the Internet of Things.

FAHAD SAMAD received the Ph.D. degree from RWTH Aachen University, specializing in network security. He is currently eagerly looking forward to academic and industrial opportunities and challenges ahead. His objective is to be actively involved and to always play a major role in shaping the technically advanced society using the skill set that he has developed over the past several years.

UMMAY FASEEHA is currently pursuing the Ph.D. degree with the Department of Computer Science, National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. She is an Assistant Professor with the Jinnah University for Women, Karachi. With a strong commitment to academic excellence, she has been involved in various research projects and has made significant contributions to the field of computer science through her teaching and research endeavors.

HAMZA AHMED is currently pursuing the Ph.D. degree in computer science with the National University of Computer and Emerging Sciences (FAST), Karachi, Pakistan. He has a rich background in academia, having a Lecturer with FAST University, for four years, before transitioning to teach A-level students with Karachi Grammar School, for two years. Alongside his doctoral studies, he is actively involved in teaching as a Visiting Faculty Member with various universities. His extensive experience in education has allowed him to contribute significantly to the academic and professional development of his students.

MUHAMMAD KHURRAM KHAN (Senior Member, IEEE) is currently a Professor of cybersecurity with the Center of Excellence in Information Assurance (CoEIA), King Saud University, Saudi Arabia. He is a Founding Member with CoEIA and has significantly contributed to its development into a leading cybersecurity research center. He is also the Founder and the CEO of the Global Foundation for Cyber Studies and Research, a cybersecurity think-tank in Washington, DC, USA. His expertise has shaped cyber policy works for the G20 and he has been involved in significant investment evaluations in cybersecurity startups. His research interests include cybersecurity, the IoT security, cloud computing security, digital authentication, and multimedia security. He is a fellow of IET and BCS and received numerous awards for his research and innovation. He serves as the Editor-in-Chief for *Telecommunication Systems* and *Cyber Insights Magazine*. He is on the editorial board of numerous prestigious journals. With over 450 research papers and ten U.S./PCT patents, his contributions to the field are widely recognized.